

NTRE

Key Encapsulation Mechanism

Algorithm Specification

(Version 1.0)

Rongmao Chen, Jinrong Chen, Yi Wang, Xinyi Huang, Yutong Bian
Yumin Chen, Zhichao Yang, Kang Lin, Jiang Tan, Wei Yan

June 30, 2026

Contents

1	Algorithm Description	1
1.1	Notation Definition	1
1.2	Auxiliary Algorithms	2
1.3	Algorithmic Procedures	4
1.3.1	Specification of NTRE.CPAPKE	5
1.3.2	Specification of NTRE.CCAKEM	6
1.4	NTRE Parameter Sets	6
2	Design Rationale	8
2.1	Main Design Ideas	8
2.2	Rationale for Parameter Selection	10
2.3	Algorithm Failure Probability	11
3	Security Statement and Analysis	14
3.1	Preliminaries	14
3.1.1	Cryptographic Primitives	14
3.1.2	The (Quantum-Accessible) Random Oracle Model	17
3.1.3	The Compressed Oracle Technique	17
3.1.4	The One-Way to Hiding Lemma	18
3.1.5	Difference Lemma	20
3.2	Theoretical Security	20
3.2.1	Security of NTRE.CPAPKE	20
3.2.2	ROM Security of NTRE.CCAKEM	21
3.2.3	Auxiliary Tools for the QROM Proof	25
3.2.4	QROM Security of NTRE.CCAKEM	27
3.3	Concrete Security	34
4	Performance Evaluation	36
4.1	Performance Analysis	36
4.2	Performance Characteristics	38
5	Statement of Key Features	39

1 Algorithm Description

NTRE is an IND-CCA2-secure key encapsulation mechanism (KEM), which security rests on the $\mathcal{R}$ -NTRU $_{\eta}$ assumption and the $\mathcal{R}$ -LWE2 $_{\eta}$ assumption. The design of NTRE follows two steps:

- First, we propose an OW-CPA-secure public-key encryption (PKE) scheme, called NTRE.CPAPKE.
- Then, through a PKE-to-KEM transform, this scheme is lifted to an IND-CCA2-secure KEM, called NTRE.CCAKEM.

Compared with existing schemes, NTRE has two notable features:

- Unlike post-quantum schemes such as ML-KEM [16] and HQC, which require the underlying PKE to have negligible *worst-case* decryption error, NTRE needs only negligible *average-case* decryption error from its underlying PKE, NTRE.CPAPKE. This allows NTRE to select shorter parameters.
- Unlike other NTRU-based schemes such as NTRU and NTRU+ [18], where the underlying PKE directly encrypts a polynomial with coefficients from some distribution, NTRE.CPAPKE encrypts a uniform binary string from $\{0, 1\}^n$. This gives NTRE more flexibility in computation.

In the following, Section 1.1 defines the notation used throughout the document. Section 1.2 then constructs the auxiliary algorithms required by NTRE. After that, Section 1.3 presents the detailed specification of NTRE.CPAPKE and NTRE.CCAKEM. Finally, Section 1.4 presents the parameter sets for each security level.

1.1 Notation Definition

We write a function H with domain $\mathcal{X}$ and codomain $\mathcal{Y}$ as $H : \mathcal{X} \rightarrow \mathcal{Y}$, and denote the set of all such functions by Ω_H . The cardinality of a set $\mathcal{S}$ is $|\mathcal{S}|$. Let $\mathbb{Z}_q$ be the set $\{-\frac{q-1}{2}, \dots, \frac{q-1}{2}\}$ for a positive odd integer q , and let $\mathcal{R}_q$ be the polynomial ring $\mathbb{Z}_q[X]/(X^n - X^{n/2} + 1)$, where the cyclotomic trinomial $X^n - X^{n/2} + 1$ has degree $n = 2^i 3^j$ for some positive integers i and j . Polynomials in $\mathcal{R}_q$ are written in bold such as $\mathbf{f}$, and $\mathbf{f}_i$ is the i -th coefficient of $\mathbf{f}$, where $\mathbf{f} = \sum_{i=0}^{n-1} \mathbf{f}_i X^i$ with $\mathbf{f}_i \in \mathbb{Z}_q$. An element $\mathbf{f} \in \mathcal{R}_q$ can be uniquely identified with its coefficient vector $\vec{v} = (\mathbf{f}_0, \dots, \mathbf{f}_{n-1}) \in \mathbb{Z}_q^n$, and we write $\mathbf{f} := \vec{v}$ to mean that $\mathbf{f}$ has coefficients given by $\vec{v}$. For $\mathbf{f}, \mathbf{g} \in \mathcal{R}_q$, $\mathbf{f} + \mathbf{g}$ and $\mathbf{f} \cdot \mathbf{g}$ denote addition and multiplication in $\mathcal{R}_q$; for vectors $\vec{v}, \vec{w}$, $\vec{v} \odot \vec{w}$ is component-wise multiplication.

The symbol ψ_1 denotes the centered binomial distribution (CBD) B_{η} with parameter $\eta = 1$, where the variable takes values in $\{-1, 0, 1\}$ with probabilities $\Pr[-1] = \Pr[1] = 0.25$ and $\Pr[0] = 0.5$. For a finite set $\mathcal{S}$, $r \leftarrow \$ \mathcal{S}$ means r is drawn uniformly from $\mathcal{S}$; for a distribution ψ , $r \leftarrow \$ \psi$ means r is sampled according to ψ . For a probabilistic (resp. deterministic) algorithm A on input x , we write $y \leftarrow A(x)$ (resp. $y := A(x)$) to denote that A outputs y . An oracle algorithm with classical (resp. quantum) access to an oracle H is denoted by A^H (resp. $A^{|H\rangle}$).

For an n -tuple $T = (t_0, \dots, t_{n-1})$, $T[i]$ denotes the $(i+1)$ -st element t_i . The notation $[a = b]$ denotes the indicator function that outputs 1 if $a = b$ and 0 otherwise. The security parameter is denoted by 1^λ , and PPT stands for *probabilistic polynomial time*. Unless stated otherwise, $\log$ is the base-2 logarithm.

NTT and NTT^{-1} Transforms. The number-theoretic transform (NTT) is an efficient approach for multiplication in $\mathcal{R}_q$, comprising two transforms:

- The forward NTT (denoted as NTT): It maps a polynomial $\mathbf{f} = \sum_{i=0}^{n-1} \mathbf{f}_i X^i \in \mathcal{R}_q$ to an n -dimensional vector $\hat{\mathbf{f}} \in \mathbb{Z}_q^d$.
- The inverse NTT (denoted as NTT^{-1}): It recovers $\mathbf{f}$ from its NTT representation $\hat{\mathbf{f}}$.

We write $\hat{\mathbf{f}} := \text{NTT}(\mathbf{f})$ and $\mathbf{f} := \text{NTT}^{-1}(\hat{\mathbf{f}})$. In the NTT domain, polynomial multiplication reduces to component-wise vector multiplication:

$$\mathbf{f} \cdot \mathbf{g} = \text{NTT}^{-1}(\hat{\mathbf{f}} \odot \hat{\mathbf{g}}) \quad \text{for any } \mathbf{f}, \mathbf{g} \in \mathcal{R}_q,$$

where $\odot$ denotes the component-wise product of vectors.

1.2 Auxiliary Algorithms

To describe the complete workflow of NTRE clearly, we first introduce several auxiliary algorithms that handle data operations within NTRE. These include:

- Byte and bit conversion algorithms (**Bytes2Bits** and **Bits2Bytes**, Algorithms 1 and 2): Note that data in algorithms is typically input and output as bytes, while some internal operations require bit-wise manipulation. To this end, **Bytes2Bits** converts a byte array into a bit array, and **Bits2Bytes** performs the reverse conversion.
- Centered binomial distribution sampling algorithms (CBD_1 and CBD'_1 , Algorithms 3 and 4): Given $n/4$ bytes, both algorithms output a polynomial $\mathbf{f} \in \mathcal{R}_q$ with coefficients in $\{-1, 0, 1\}$. When the input bytes are uniformly random, $\mathbf{f}$ can be regarded as drawn from the distribution ψ_1^n . Specially, in CBD'_1 , the first $n/2$ bytes of the input can be recovered from $\mathbf{f} \bmod 2$.
- Polynomial sampling algorithm (**Sample**, Algorithm 5): It samples a polynomial from $\mathcal{R}_q$ whose coefficients follow the distribution ψ_1 . These polynomials are used to generate keys and other variables of NTRE.
- Polynomial encoding and decoding algorithms (**Encode_q** and **Decode_q**, Algorithms 6 and 7): They used to store polynomials as bytes. **Encode_q** encodes a polynomial into a byte array, and **Decode_q** recovers the polynomial from the encoded bytes.

Specifically, these auxiliary algorithms are described as follows:

Bit and Byte Conversion Algorithms. **Bytes2Bits** converts a byte array into a bit array, while **Bits2Bytes** packs a bit array back into a byte array. Here, $\mathcal{B}$ denotes the set of all possible byte values, i.e., the set of 8-bit strings $\{0, 1\}^8$.

Algorithm 1 Bytes2Bits

Input: Byte array $B := (b_0, b_1, \dots, b_{n/8-1}) \in \mathcal{B}^{n/8}$

Output: Bit array $\beta := (\beta_0, \beta_1, \dots, \beta_{n-1}) \in \{0, 1\}^n$

- 1: **For** $i = 0, \dots, n/8 - 1$ **Do**
 - 2: $t := b_i$
 - 3: **For** $j = 0, \dots, 7$ **Do**
 - 4: $\beta_{8i+j} := t \& 1$
 - 5: $t := t \gg 1$
 - 6: **Return** $\beta := (\beta_0, \beta_1, \dots, \beta_{n-1})$
-

Algorithm 2 Bits2Bytes

Input: Bit array $\beta := (\beta_0, \beta_1, \dots, \beta_{n-1}) \in \{0, 1\}^n$
Output: Byte array $B := (b_0, b_1, \dots, b_{n/8-1}) \in \mathcal{B}^{n/8}$
1: **For** $i = 0, \dots, n/8 - 1$ **Do**
2: $b_i := 0$
3: **For** $j = 0, \dots, 7$ **Do**
4: $b_i := b_i \mid (\beta_{8i+j} \ll j)$
5: **Return** $B := (b_0, b_1, \dots, b_{n/8-1})$

Centered Binomial Distribution Sampling Algorithms. Given $n/4$ bytes (i.e., $2n$ bits), the input is split into the first n bits and the last n bits. CBD_1 obtains the n coefficients $\mathbf{f}_i$ by subtracting these bits coordinate-wise, while CBD'_1 computes each $\mathbf{f}_i$ as $\beta_i(1 - 2\beta_{n+i})$, where β_i and β_{n+i} are the corresponding bits from the two halves.

In both cases, the result is a polynomial $\mathbf{f} := (\mathbf{f}_0, \dots, \mathbf{f}_{n-1}) \in \mathcal{R}_q$ with coefficients in $\{-1, 0, 1\}$. It is not hard to verify that when the input is uniform randomly sampled, each coefficient $\mathbf{f}_i$ follows the distribution ψ_1 .

Algorithm 3 CBD_1

Input: Bit array $B := (b_0, b_1, \dots, b_{n/4-1}) \in \mathcal{B}^{n/4}$
Output: Polynomial $\mathbf{f} := (\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_{n-1})$, where $\mathbf{f}_i \in \{-1, 0, 1\}$
1: $(\beta_0, \beta_1, \dots, \beta_{n-1}) := \text{Bytes2Bits}((b_0, b_1, \dots, b_{n/8-1}))$
2: $(\beta_n, \beta_{n+1}, \dots, \beta_{2n-1}) := \text{Bytes2Bits}((b_{n/8}, b_{n/8+1}, \dots, b_{n/4-1}))$
3: **For** $i = 0, \dots, n - 1$ **Do**
4: $\mathbf{f}_i := \beta_i - \beta_{n+i}$
5: **Return** $\mathbf{f} := (\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_{n-1})$

Algorithm 4 CBD'_1

Input: Bit array $B := (b_0, b_1, \dots, b_{n/4-1}) \in \mathcal{B}^{n/4}$
Output: Polynomial $\mathbf{f} := (\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_{n-1})$, where $\mathbf{f}_i \in \{-1, 0, 1\}$
1: $(\beta_0, \beta_1, \dots, \beta_{n-1}) := \text{Bytes2Bits}((b_0, b_1, \dots, b_{n/8-1}))$
2: $(\beta_n, \beta_{n+1}, \dots, \beta_{2n-1}) := \text{Bytes2Bits}((b_{n/8}, b_{n/8+1}, \dots, b_{n/4-1}))$
3: **For** $i = 0, \dots, n - 1$ **Do**
4: $\mathbf{f}_i := \beta_i \cdot (1 - 2 \cdot \beta_{n+i})$
5: **Return** $\mathbf{f} := (\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_{n-1})$

It is also not hard to see that, in CBD'_1 , the first n bits $(\beta_0, \beta_1, \dots, \beta_{n-1})$ of the input B can be recovered by computing $\mathbf{f} \bmod 2$.

Polynomial Sampling Algorithm. `Sample` first calls a extendable-output function XOF to obtain $2n$ bits (i.e., $n/4$ bytes). From these bits, it then generates a polynomial in $\mathcal{R}_q$ whose coefficients follow the distribution ψ_1 .

Algorithm 5 Sample

Input: Dimension n of the polynomial ring $\mathcal{R}_q$

Output: Polynomial $\mathbf{f} := (\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_{n-1})$, where $\mathbf{f}_i \leftarrow \$ \psi_1$

- 1: $s \leftarrow \$ \mathcal{B}^{64}$
 - 2: $B = (b_0, b_1, \dots, b_{n/4-1}) := \text{XOF}(s, n/4)$
 - 3: $\mathbf{f} := \text{CBD}_1(B)$
 - 4: **Return** $\mathbf{f}$
-

Polynomial Encoding and Decoding Algorithms. Encode_q packs pairs of coefficients of $\mathbf{f} \in \mathcal{R}_q$ into three bytes, while Decode_q recovers the polynomial coefficients from those encoded bytes. Both algorithms assume that q is at most 12 bits.

For each pair, the first byte stores the low 8 bits of the first coefficient, the third byte stores the high 8 bits of the second coefficient, and the middle byte combines the high 4 bits of the first coefficient with the low 4 bits of the second coefficient. In all parameter sets described in this document, q is 12 bits.

Algorithm 6 Encode_q

Input: Polynomial $\mathbf{f} := (\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_{n-1}) \in \mathcal{R}_q$

Output: Byte array $B := (b_0, b_1, \dots, b_{3n/2-1})$

- 1: **For** $i = 0, \dots, n/2 - 1$ **Do**
 - 2: $t_0 := \mathbf{f}_{2i}$
 - 3: $t_1 := \mathbf{f}_{2i+1}$
 - 4: $b_{3i} := t_0 \& 0\text{xff}$
 - 5: $b_{3i+1} := (t_0 \gg 8) \mid ((t_1 \& 0\text{x0f}) \ll 4)$
 - 6: $b_{3i+2} := t_1 \gg 4$
 - 7: **Return** $B := (b_0, b_1, \dots, b_{3n/2-1})$
-

Algorithm 7 Decode_q

Input: Byte array $B := (b_0, \dots, b_{3n/2-1})$

Output: Polynomial $\mathbf{f} \in \mathcal{R}_q$

- 1: **For** $i = 0, \dots, n/2 - 1$ **Do**
 - 2: $t_0 := b_{3i}$
 - 3: $t_1 := b_{3i+1}$
 - 4: $t_2 := b_{3i+2}$
 - 5: $\mathbf{f}_{2i} := t_0 \mid ((t_1 \& 0\text{x0f}) \ll 8)$
 - 6: $\mathbf{f}_{2i+1} := (t_1 \gg 4) \mid (t_2 \ll 4)$
 - 7: **Return** $\mathbf{f} := (\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_{n-1})$
-

1.3 Algorithmic Procedures

This section provides a complete description of our NTRE scheme. We first specify the underlying PKE scheme, NTRE.CPAPKE, and then present the KEM, NTRE.CCAKEM, constructed from it.

1.3.1 Specification of NTRE.CPAPKE

NTRE.CPAPKE = (KGen, Enc, Dec) is a public-key encryption scheme parameterized by integers n and q . Unlike the underlying PKE in other NTRU-based schemes, such as NTRU and NTRU+, which directly encrypt a polynomial in $\mathcal{R}_q$, NTRE.CPAPKE encrypts a binary string $m \in \{0, 1\}^n$ as the message. Using the notation defined in Section 1.1 and the auxiliary algorithms introduced in Section 1.2, we present the key generation, encryption, and decryption algorithms of NTRE.CPAPKE in Algorithms 8, 9, and 10, respectively. A higher-level description is provided in the comments within the algorithms.

Algorithm 8 NTRE.CPAPKE.KGen(1^λ): key generation

Output: Public key $\text{pk} \in \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/8}$

Output: Private key $\text{sk} \in \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/8}$

```

1: Repeat
2:    $\mathbf{f}' \leftarrow \text{Sample}(n)$   $\triangleright \mathbf{f}' \leftarrow \$ \psi_1^n$ 
3:    $\hat{\mathbf{f}} := \text{NTT}(2 \cdot \mathbf{f}' + 1)$   $\triangleright \mathbf{f} := 2 \cdot \mathbf{f}' + 1$ 
4: Until  $\mathbf{f}$  is invertible in  $\mathcal{R}_q$ 
5:  $\mathbf{g}' \leftarrow \text{Sample}(n)$   $\triangleright \mathbf{g}' \leftarrow \$ \psi_1^n$ 
6:  $\hat{\mathbf{g}} := \text{NTT}(2 \cdot \mathbf{g}')$   $\triangleright \mathbf{g} := 2 \cdot \mathbf{g}'$ 
7:  $\hat{\mathbf{h}} := \hat{\mathbf{g}} \odot \hat{\mathbf{f}}^{-1}$ 
8:  $\text{pk} := \text{Encode}_q(\hat{\mathbf{h}})$   $\triangleright \text{pk} := \mathbf{h} (= \mathbf{g} \cdot \mathbf{f}^{-1})$ 
9:  $\text{sk} := \text{Encode}_q(\hat{\mathbf{f}})$   $\triangleright \text{sk} := \mathbf{f}$ 
10: Return (pk, sk)

```

Algorithm 9 NTRE.CPAPKE.Enc(pk, m; r): encryption

Input: Public key $\text{pk} \in \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/8}$

Input: Plaintext $m := (m_0, m_1, \dots, m_{n/8-1}) \in \mathcal{B}^{n/8}$

Input: Random coin $r := (r_0, r_1, \dots, r_{3n/8-1}) \leftarrow \$ \mathcal{B}^{3n/8}$

Output: Ciphertext $c \in \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/8}$

```

1:  $\mathbf{r} := \text{CBD}_1((r_1, \dots, r_{n/4-1}))$   $\triangleright \mathbf{r} \leftarrow \$ \psi_1^n$ 
2:  $\hat{\mathbf{r}} := \text{NTT}(\mathbf{r})$ 
3:  $\mathbf{e} := \text{CBD}'_1(m \parallel (r_{n/4}, \dots, r_{3n/8-1}))$   $\triangleright r' \leftarrow \$ \{0, 1\}^n, \mathbf{e} := \text{CBD}'_1(m, r')$ 
4:  $\hat{\mathbf{e}} := \text{NTT}(\mathbf{e})$ 
5:  $\hat{\mathbf{h}} := \text{Decode}_q(\text{pk})$ 
6:  $\hat{\mathbf{c}} := \hat{\mathbf{h}} \odot \hat{\mathbf{r}} + \hat{\mathbf{e}}$   $\triangleright \mathbf{c} := \mathbf{h} \cdot \mathbf{r} + \mathbf{e}$ 
7:  $c := \text{Encode}_q(\hat{\mathbf{c}})$ 
8: Return c

```

Algorithm 10 NTRE.CPAPKE.Dec(sk, c): decryption

Input: Private key $\text{sk} \in \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/8}$

Input: Ciphertext $c \in \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/8}$

Output: Plaintext $m \in \mathcal{B}^{n/8}$

```

1:  $\hat{\mathbf{f}} := \text{Decode}_q(\text{sk}), \hat{\mathbf{c}} := \text{Decode}_q(c)$ 
2:  $\mathbf{m} := \text{NTT}^{-1}(\hat{\mathbf{c}} \odot \hat{\mathbf{f}}) \bmod 2$   $\triangleright \mathbf{m} := (c \cdot \mathbf{f} \bmod \pm q) \bmod 2$ 
3:  $m := \text{Bits2Bytes}(\mathbf{m})$ 
4: Return m

```

1.3.2 Specification of NTRE.CCAKEM

We now construct $\text{NTRE.CCAKEM} = (\text{KGen}, \text{Encaps}, \text{Decaps})$, a probabilistic key encapsulation scheme obtained by applying a slight variant of the FOAC transform [11] (which we call FOAC' [7]) to NTRE.CPAPKE . The key generation, encapsulation, and decapsulation algorithms of NTRE.CCAKEM are specified in Algorithms 11, 12, and 13, respectively. The hash functions F , G , and H are modeled as random oracles; their input and output spaces are indicated in the comments of the algorithms.

Algorithm 11 $\text{NTRE.CCAKEM.KGen}(1^\lambda)$: key generation

Output: Public key $\text{pk} \in \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/8}$

Output: Private key $\text{sk} \in \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/4 + 32}$

1: $(\text{pk}, \text{sk}') \leftarrow \text{NTRE.CPAPKE.KGen}(1^\lambda)$

2: $\text{sk} := \text{sk}' \parallel \text{pk} \parallel G(\text{pk})$

3: **Return** (pk, sk)

$\triangleright$ Algorithm 8
 $\triangleright G : \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/8} \rightarrow \mathcal{B}^{32}$

Algorithm 12 $\text{NTRE.CCAKEM.Encaps}(\text{pk})$: encapsulation

Input: Public key $\text{pk} \in \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/8}$

Output: Shared key $K \in \mathcal{B}^{64}$

Output: Ciphertext $c \in \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/8}$

1: $r \leftarrow \$ \mathcal{B}^{\lceil n/16 \rceil}$

2: $(m_1, \rho, K) := H(r \parallel G(\text{pk}))$

3: $m_2 := r \oplus F(m_1)$

4: $c := \text{NTRE.CPAPKE.Enc}(\text{pk}, m_1 \parallel m_2; \rho)$

5: **Return** (K, c)

$\triangleright H : \mathcal{B}^{\lceil n/16 \rceil} \times \mathcal{B}^{32} \rightarrow \mathcal{B}^{\lceil n/16 \rceil} \times \mathcal{B}^{3n/8} \times \mathcal{B}^{64}$
 $\triangleright F : \mathcal{B}^{\lceil n/16 \rceil} \rightarrow \mathcal{B}^{\lceil n/16 \rceil}$
 $\triangleright$ Algorithm 9

Algorithm 13 $\text{NTRE.CCAKEM.Decaps}(\text{sk}, c)$: decapsulation

Input: Private key $\text{sk} \in \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/4 + 32}$

Input: Ciphertext $c \in \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/8}$

Output: Shared key $K \in \mathcal{B}^{64}$

1: **Parse** $\text{sk} = (\text{sk}', \text{pk}, h) \in \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/8} \times \mathcal{B}^{\lceil \log_2 q \rceil \cdot n/8} \times \mathcal{B}^{32}$

2: $m_1 \parallel m_2 := \text{NTRE.CPAPKE.Dec}(\text{sk}', c_1) \in \mathcal{B}^{\lceil n/16 \rceil} \times \mathcal{B}^{\lceil n/16 \rceil}$

3: $r := m_2 \oplus F(m_1)$

4: $(m'_1, \rho, K) := H(r \parallel h)$

5: $c' := \text{NTRE.CPAPKE.Enc}(\text{pk}, m_1 \parallel m_2; \rho)$

6: **If** $m_1 = m'_1$ **and** $c = c'$ **Then**

7: **Return** K

8: **Return** $\perp$

$\triangleright$ Algorithm 10

1.4 NTRE Parameter Sets

For different security levels, we define three parameter sets for NTRE, which we refer to as NTRE-2593-648, NTRE-2917-1296, and NTRE-3457-2304, as shown in Table 1. Here n and q denote the degree and modulus of the polynomial ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n - X^{n/2} + 1)$, respectively. In the table, we also list the decapsulation failure probability δ of a valid

Table 1: Parameter Sets of NTRE

	n	q	δ	Security Level	
				Classical	Quantum
NTRE-2593-648	648	2593	2^{-549}	132	118
NTRE-2917-1296	1296	2917	2^{-370}	290	259
NTRE-3457-2304	2304	3457	2^{-298}	584	513

NTRE.CCAKEM ciphertext for each parameter set. The rationale behind these parameter choices and a detailed analysis of the failure probability are presented in Sections 2.2 and 2.3, respectively.

Instantiations of the Extendable-Output Function XOF, Hash Functions F, G, H, and Random Sampling. Note that the extendable-output function XOF, the hash functions F, G, H, and the random sampling method used in NTRE have not yet been instantiated. According to Section 3.4 of the “Submission Requirements for Public-Key Cryptographic Algorithms” of the NGCC, we instantiate all these symmetric primitives using the API provided by ICCS as follows:

- For the extendable-output function XOF and the hash functions F and H: instantiated with the provided `pseudoXOF` (built from KDF-SM3), by prepending a distinct one-byte prefix to the input message (“0x00” for XOF, “0x01” for F, “0x02” for H).
- For the hash function G: instantiated with the provided `sm3hash` (built from SM3), which outputs 32 bytes.
- For random sampling: instantiated with the provided Deterministic Random Number Generator (DRNG).

2 Design Rationale

NTRE is a key encapsulation mechanism (KEM) built upon the well-established hardness of the $\mathcal{R}$ -NTRU $_\eta$ and $\mathcal{R}$ -LWE2 $_\eta$ problems, designed to meet the three NGCC security levels while offering high speed and compact parameters. Section 2.1 introduces the main design ideas behind NTRE, Section 2.2 explains the rationale for the parameter choices, and Section 2.3 analyzes the failure probability together with its impact on security.

2.1 Main Design Ideas

The design of NTRE follows the prevailing approach for post-quantum KEMs, i.e., first building a CPA-secure PKE and then lifting it to an IND-CCA2-secure KEM via a PKE-to-KEM transform.

Overall Design of NTRE.CPAPKE. The underlying CPA-secure PKE, NTRE.CPAPKE = (KGen, Enc, Dec), is rooted in the NTRU tradition, where the three algorithms that form NTRE.CPAPKE work as follows:

- **Key generation (KGen):** On input a security parameter 1^λ , this algorithm outputs a key pair $(\mathbf{pk}, \mathbf{sk})$, where $\mathbf{sk} := \mathbf{f} = p\mathbf{f}' + 1$ and $\mathbf{pk} := \mathbf{h} = p\mathbf{g}/\mathbf{f}$, with a small prime p that is much smaller than q . Here, the coefficients of $\mathbf{f}', \mathbf{g} \in \mathcal{R}_q$ are drawn from ψ_1 , and $\mathbf{f}$ is required to be invertible in $\mathcal{R}_q$.
- **Encryption (Enc):** On input a polynomial $\mathbf{m} \in \mathcal{R}_q$ with coefficients in $\{-1, 0, 1\}$ and the public key $\mathbf{pk} = \mathbf{h}$, this algorithm produces the ciphertext $\mathbf{c} = \mathbf{r} \cdot \mathbf{h} + \mathbf{m} \in \mathcal{R}_q$, where the coefficients of the randomness $\mathbf{r} \in \mathcal{R}_q$ are also sampled from ψ_1 .
- **Decryption (Dec):** On input the ciphertext $\mathbf{c} \in \mathcal{R}_q$ and the private key $\mathbf{sk} = \mathbf{f}$, the algorithm recovers the plaintext $\mathbf{m}$ by computing $\mathbf{m} := \mathbf{f} \cdot \mathbf{c} \bmod^{\pm p}$, where the product $\mathbf{f} \cdot \mathbf{c}$ is taken in $\mathcal{R}_q$.

When q is sufficiently large, we have

$$\mathbf{f} \cdot \mathbf{c} = \mathbf{f} \cdot (\mathbf{r} \cdot \mathbf{h} + \mathbf{m}) = p\mathbf{g} \cdot \mathbf{r} + (p\mathbf{f}' + 1) \cdot \mathbf{m} = p(\mathbf{g} \cdot \mathbf{r} + \mathbf{f}' \cdot \mathbf{m}) + \mathbf{m} \quad (\text{in } \mathcal{R}_q) . \quad (1)$$

Since $p(\mathbf{g} \cdot \mathbf{r} + \mathbf{f}' \cdot \mathbf{m})$ vanishes modulo p and the coefficients of $\mathbf{m}$ lie in $\mathbb{Z}_p$, we obtain

$$(\mathbf{f} \cdot \mathbf{c} \bmod^{\pm q}) \bmod^{\pm p} = \mathbf{m} , \quad (2)$$

which recovers the plaintext $\mathbf{m} \in \mathcal{R}_q$.

Taking $p = 2$ for Compact Parameters. Note that the successful recovery of $\mathbf{m}$ above relies on q being large enough that the computation in (1) never causes a coefficient to “wrap around”. Equivalently, every coefficient must stay within $[-(q-1)/2, (q-1)/2]$ in absolute value. If this condition is violated, the subsequent $\bmod^{\pm p}$ operation in (2) will produce a wrong $\mathbf{m}$ and thus a decryption error.

As shown in Section 2.3, the decryption error probability of NTRE.CPAPKE for a random $\mathbf{m}$ is jointly determined by n , p , and q , where n and q are the degree and modulus of the polynomial ring $\mathcal{R}_q$, respectively. A smaller p therefore permits smaller n and q while still meeting the NGCC security requirements and keeping the failure probability negligible, as explained in Section 2.2. This, in turn, significantly reduces the sizes of the

key pair $(\mathbf{pk}, \mathbf{sk})$ and the ciphertext $\mathbf{c}$ in NTRE.CPAPKE. To obtain the most compact parameter set for NTRE, we take p to be the smallest prime, namely 2.

Encoding $m \in \{0, 1\}^n$ into the Polynomial $\mathbf{m} \in \mathcal{R}_q$. However, when $p = 2$, Eq. (1) cannot directly recover a polynomial $\mathbf{m}$ with coefficients in $\{-1, 0, 1\}$, because reduction modulo 2 always yields a bit. Consequently, the plaintext space now must be $\{0, 1\}^n$.

Inspired by NTRU-A [11], to encode a message $m \in \{0, 1\}^n$ into a polynomial $\mathbf{m} \in \mathcal{R}_q$ with coefficients in $\{-1, 0, 1\}$ suitable for encryption, we design the sampling algorithm CBD'_1 (Algorithm 4), then pick a random $r \in \{0, 1\}^n$ and set $\mathbf{m} := \text{CBD}'_1(m \| r) \in \mathcal{R}_q$. Note that the coefficients of $\mathbf{m}$ then lie in $\{-1, 0, 1\}$, and when m is uniformly random, they follow the distribution ψ_1 , which meets the requirement of the $\mathcal{R}$ -LWE $_{2,\eta}$ assumption (with $\eta = \psi_1^n$) on the message distribution, as discussed in Section 3.2.1. Moreover, the original message m can be recovered by computing $\mathbf{m} \bmod 2$, which matches the decryption step in (2).

KEM Construction from Average-Case Decryption Error. Most post-quantum KEMs, including the NIST-standardized ML-KEM and the NIST candidate HQC, are built from PKEs via an FO-like transform. However, such transform requires the underlying PKE to have negligible *worst-case* decryption error. Formally, for $\text{PKE} = (\text{KGen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ and randomness space $\mathcal{R}$, the required condition is

$$\delta_{wc} := \mathbb{E}_{(\mathbf{pk}, \mathbf{sk}) \leftarrow \text{Gen}(1^\lambda)} \left[\max_{m \in \mathcal{M}} \Pr_{\rho \leftarrow \mathcal{R}} [\text{Dec}(\mathbf{sk}, \text{Enc}(\mathbf{pk}, m; \rho)) \neq m] \right]$$

being negligible. Thus, the designer must control the error for *every* possible message, which is often nontrivial. For NTRU-based PKEs, reducing δ_{wc} by increasing q makes lattice attacks more effective, weakening security [11].

In contrast, the *average-case* decryption error

$$\delta_{ac} := \mathbb{E}_{(\mathbf{pk}, \mathbf{sk}) \leftarrow \text{Gen}(1^\lambda)} \left[\Pr_{m \leftarrow \mathcal{M}, \rho \leftarrow \mathcal{R}} [\text{Dec}(\mathbf{sk}, \text{Enc}(\mathbf{pk}, m; \rho)) \neq m] \right]$$

is far more designer-friendly: it requires only negligible failure on *random* messages, thereby relaxing parameter constraints. To take advantage of this property, we construct NTRE.CCAKEM via the FOAC' transform (Figure 1), a slight variant of $\text{FOAC} := \text{FO}_m^\perp \circ \text{ACWC}$ from [11]. In addition to relying only on the *average-case* decryption error, FOAC' eliminates the γ -spread condition required by the original FOAC, which simplifies the construction and improves efficiency. It also detects *invalid* ciphertexts that FOAC would miss, further enhancing reliability.

Additional Hashing. Following the FOAC' transform (Figure 1) and ML-KEM, in NTRE.CCAKEM we hash the public key $\mathbf{pk}$ with an extra hash function G and feed the result into the generation of the randomness (m_1, ρ) and the shared key K (line 2 of Algorithm 12), which adds protection against a certain class of multi-target attacks. Although these hashing steps are not necessary for the security reduction (see Sections 3.2.2 and 3.2.4), they will make NTRE.CCAKEM more robust.

Using the NTT. Using the number-theoretic transform (NTT) to multiply polynomials in $\mathcal{R}_q$ is now standard practice in lattice-based cryptography: in the NTT domain, multiplication becomes much faster. To fully benefit from the NTT, all polynomials in NTRE are taken from the polynomial-friendly ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n - X^{n/2} + 1)$, where $n = 2^i 3^j$ for some positive integers i, j and the prime q satisfies $q \equiv 1 \pmod{n}$. To avoid

$\text{Gen}(1^\lambda)$	$\text{Decaps}(\text{sk}, c)$
1 : $(\text{pk}, \text{sk}) \leftarrow \text{Gen}'(1^\lambda)$	1 : $m_1 \ m_2 := \text{Dec}'(\text{sk}, c)$
2 : return (pk, sk)	2 : if $m_1 \ m_2 = \perp$ then : return $\perp$
$\text{Encaps}(\text{pk})$	3 : $r := m_2 \oplus F(m_1)$
1 : $r \leftarrow \$ \mathcal{M}_2$	4 : $(m'_1, \rho, k) := H(r)$
2 : $(m_1, \rho, k) := H(r)$	5 : if $m_1 = m'_1$ and
3 : $m_2 := r \oplus F(m_1)$	$\text{Enc}'(\text{pk}, m'_1 \ m_2; \rho) = c$ then
4 : $c := \text{Enc}'(\text{pk}, m_1 \ m_2; \rho)$	return k
5 : return (k, c)	6 : return $\perp$

Figure 1: $\text{KEM} := \text{FOAC}'[\text{PKE}', F, H]$, where $F : \mathcal{M}_1 \rightarrow \mathcal{M}_2$ and $H : \mathcal{M}_2 \rightarrow \mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}$ are hash functions. Here, $\mathcal{M}_1 \times \mathcal{M}_2$ is the message space of $\text{PKE}' = (\text{Gen}', \text{Enc}', \text{Dec}')$, $\mathcal{R}$ is the randomness space of Enc' , and $\mathcal{K}$ is the key space of the derived KEM KEM .

the overhead of repeated NTT conversions, the public-key and secret-key polynomials are stored directly in their NTT representation.

2.2 Rationale for Parameter Selection

The security strength of NTRE is jointly determined by the degree n and modulus q of the polynomial ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n - X^{n/2} + 1)$. We define three parameter sets for NTRE , corresponding to the three security levels required by the NGCC, as listed in Table 1. The criteria for choosing n and q under each security level are as follows:

- (1) First, the parameters must satisfy a few basic requirements: q is a prime, $n = 2^i 3^j$ for some integers i, j , and $q \equiv 1 \pmod{n}$, which makes $\mathcal{R}_q$ NTT-friendly. Additionally, q is set to 12 bits, i.e., in the range 2^{11} to 2^{12} . This is a standard engineering trade-off between efficiency and security, consistent with lattice-based post-quantum KEMs such as ML-KEM and NTRU+.
- (2) Second, n and q must withstand the best known attacks against the $\mathcal{R}\text{-LWE}_\eta$ assumption¹ and the $\mathcal{R}\text{-NTRU}_\eta$ assumption. We assess this using the Lattice Estimator and the NTRU Estimator, which give classical and quantum security estimates for these two assumptions. Both classical and quantum security levels must satisfy the NGCC requirements.
- (3) Finally, for the chosen n and q , the decapsulation error probability of the scheme must be negligible.

Regarding criterion (1), we restrict q to Pierpont primes of the form $q = 2^u 3^v + 1$. For 12-bit primes, the available choices are $q = 2593, 2917, 3457, 3889$. For criterion (2), we run the Lattice Estimator and the NTRU Estimator for each of these four values of q and for all positive integers n dividing $q - 1$, obtaining classical and quantum security estimates under the $\mathcal{R}\text{-LWE}_\eta$ and $\mathcal{R}\text{-NTRU}_\eta$ assumptions. To verify condition (3), we

¹Although NTRE is based on the $\mathcal{R}\text{-LWE}2_\eta$ assumption rather than $\mathcal{R}\text{-LWE}_\eta$, as analyzed in [11], the concrete hardness of the two problems is equivalent.

compute the corresponding decapsulation error probability for every (n, q) pair using the analysis of Section 2.3. The relevant results are shown in Table 2².

Table 2: Estimated Security (bits) for $\eta = \psi_1$ and δ with n, q

q	n	Classical Security Level		Quantum Security Level		δ
		$\mathcal{R}\text{-LWE}_\eta$	$\mathcal{R}\text{-NTRU}_\eta$	$\mathcal{R}\text{-LWE}_\eta$	$\mathcal{R}\text{-NTRU}_\eta$	
2593	648	132	139	118	122	2^{-549}
2593	864	184	196	165	172	2^{-427}
2593	1152	257	274	229	241	2^{-328}
2593	1296	293	314	261	276	2^{-293}
2593	1728	405	435	361	383	2^{-222}
2593	2592	685	570	602	502	2^{-147}
2917	648	130	136	116	120	2^{-684}
2917	972	209	222	186	195	2^{-482}
2917	1296	290	310	259	272	2^{-370}
2917	1458	330	355	295	312	2^{-331}
2917	1944	456	491	407	432	2^{-250}
2917	2916	771	512	678	424	2^{-166}
3457	768	156	164	139	144	2^{-811}
3457	864	179	189	160	166	2^{-735}
3457	1152	248	266	222	233	2^{-572}
3457	1536	344	369	307	325	2^{-439}
3457	1728	393	423	351	371	2^{-393}
3457	2304	584	584	514	513	2^{-298}
3889	864	176	186	157	164	2^{-913}
3889	972	201	214	180	188	2^{-827}
3889	1296	281	300	250	264	2^{-644}
3889	1728	389	417	347	367	2^{-495}
3889	1944	443	477	395	419	2^{-443}
3889	2592	658	535	579	471	2^{-335}

Based on Table 2 and the three conditions discussed above, we select the parameter sets shown in Table 1 for the three NGCC security levels.

2.3 Algorithm Failure Probability

As illustrated in Figure 1, a decapsulation error in NTRE.CCAKEM (as defined in Definition 3.8) implies that for a random $r \leftarrow \$\mathcal{M}_2$, we have

$$\text{Dec}'(\text{sk}, \text{Enc}'(\text{pk}, m_1 \| m_2; \rho)) \neq m_1 \| m_2,$$

where $(m_1, \rho, \cdot) := H(r)$ and $m_2 := r \oplus F(m_1)$. When H and F are modeled as random oracles, a uniformly chosen r makes m_1 , ρ , and m_2 uniformly distributed in their respective

²Values of n that are too small (yielding classical security below 128 bits or quantum security below 80 bits) or unnecessarily large (exceeding the required security level) are omitted from the table.

spaces. Consequently, the decapsulation failure probability of NTRE.CCAKEM is bounded by the average-case decryption error δ of the underlying NTRE.CPAPKE.

In the following, we analyze the average-case decryption error δ of NTRE.CPAPKE with respect to the distribution ψ_1 , following the template of [15]. The decryption algorithm takes a ciphertext of the form $\mathbf{c} = 2\mathbf{g}/\mathbf{f} \cdot \mathbf{r} + \mathbf{m}$ and multiplies it by $\mathbf{f} = 2\mathbf{f}' + 1$ to obtain

$$2\mathbf{g}\mathbf{r} + 2\mathbf{f}'\mathbf{m} + \mathbf{m} = 2(\mathbf{g}\mathbf{r} + \mathbf{f}'\mathbf{m}) + \mathbf{m} \quad (3)$$

For correctness, we require that all coefficients in (3) have absolute value less than $q/2$. That is,

$$|2(\mathbf{g}\mathbf{r} + \mathbf{f}'\mathbf{m}) + \mathbf{m}|_i < \frac{q}{2}. \quad (4)$$

Since the coefficients of $\mathbf{m}$ have absolute value at most 1, we need $|\mathbf{g}\mathbf{r} + \mathbf{f}'\mathbf{m}|_i < (q-2)/4$ in order to avoid decryption errors.

The degree- i coefficient of $\mathbf{g}\mathbf{r} + \mathbf{f}'\mathbf{m}$ can be modeled as the sum of n independent random variables $c = ab + b'(a + a')$, whose distribution is listed in Table 3, where a, b, b', a' are independent and have the same distribution ψ_1 .

We illustrate this result with a simple example. Let $\mathbf{a}$ and $\mathbf{b}$ be in the ring $\mathbb{Z}_q[X]/(X^6 - X^3 + 1)$, where $\mathbf{a} = \sum_{i=0}^5 \mathbf{a}_i X^i$ and $\mathbf{b} = \sum_{i=0}^5 \mathbf{b}_i X^i$. Then, their product $\mathbf{c} = \sum_{i=0}^5 \mathbf{c}_i X^i$ can be represented by the following matrix-vector product:

$$\begin{pmatrix} \mathbf{a}_0 & -\mathbf{a}_5 & -\mathbf{a}_4 & -\mathbf{a}_3 & -\mathbf{a}_2 - \mathbf{a}_5 & -\mathbf{a}_1 - \mathbf{a}_4 \\ \mathbf{a}_1 & \mathbf{a}_0 & -\mathbf{a}_5 & -\mathbf{a}_4 & -\mathbf{a}_3 & -\mathbf{a}_2 - \mathbf{a}_5 \\ \mathbf{a}_2 & \mathbf{a}_1 & \mathbf{a}_0 & -\mathbf{a}_5 & -\mathbf{a}_4 & -\mathbf{a}_3 \\ \mathbf{a}_3 & \mathbf{a}_2 + \mathbf{a}_5 & \mathbf{a}_1 + \mathbf{a}_4 & \mathbf{a}_0 + \mathbf{a}_3 & \mathbf{a}_2 & \mathbf{a}_1 \\ \mathbf{a}_4 & \mathbf{a}_3 & \mathbf{a}_2 + \mathbf{a}_5 & \mathbf{a}_1 + \mathbf{a}_4 & \mathbf{a}_0 + \mathbf{a}_3 & \mathbf{a}_2 \\ \mathbf{a}_5 & \mathbf{a}_4 & \mathbf{a}_3 & \mathbf{a}_2 + \mathbf{a}_5 & \mathbf{a}_1 + \mathbf{a}_4 & \mathbf{a}_0 + \mathbf{a}_3 \end{pmatrix} \cdot \begin{pmatrix} \mathbf{b}_0 \\ \mathbf{b}_1 \\ \mathbf{b}_2 \\ \mathbf{b}_3 \\ \mathbf{b}_4 \\ \mathbf{b}_5 \end{pmatrix} = \begin{pmatrix} \mathbf{c}_0 \\ \mathbf{c}_1 \\ \mathbf{c}_2 \\ \mathbf{c}_3 \\ \mathbf{c}_4 \\ \mathbf{c}_5 \end{pmatrix}. \quad (5)$$

For the bottom part of $\mathbf{c}$, $\mathbf{c}_i$ can be modeled as the sum of three independent random variables

$$c = ab + b'(a + a'); \quad (6)$$

whereas, for the top part of $\mathbf{c}$, $\mathbf{c}_i$ can be modeled as the sum of $(3 - i - 1)$ independent random variables of the form in (6) and $(i + 1)$ independent random variables of the form

$$c = ab + a'b'. \quad (7)$$

Since the distribution of the bottom part is wider, we will conservatively use the random variable listed in (6) to model the distributions of $\mathbf{g}\mathbf{r}$ and $\mathbf{f}'\mathbf{m}$.

Let $\theta(X) = \sum_{i=-3}^3 \theta_i X^i$ be the generating function of the distribution of c listed in Table 3, and let $\rho(X)$ be the generating function of the degree- i coefficients of $\mathbf{g}\mathbf{r} + \mathbf{f}'\mathbf{m}$; then, by convolution, we obtain

$$\rho(X) = \theta(X)^n. \quad (8)$$

Thus, the probability that any coefficient of $\mathbf{g}\mathbf{r} + \mathbf{f}'\mathbf{m}$ is greater than $q/2$ is

$$2 \cdot \sum_{i=(q-2)/4}^{3n} \rho_i, \quad (9)$$

and we obtain the following lemma.

Lemma 2.1. *When $\mathbf{g}, \mathbf{r}, \mathbf{f}'$ and $\mathbf{m}$ are chosen from ψ_1 , the probability of a decryption error of NTRE.CPAPKE is at most*

$$2n \cdot \sum_{i=(q-2)/4}^{3n} \rho_i,$$

where ρ_i are as in (9).

Given our parameter selection, the average-case decryption failure probability of the underlying NTRE.CPAPKE is negligible; consequently, the decapsulation failure probability of NTRE.CCAKEM is also negligible. The concrete results are listed in Table 1.

± 3	± 2	± 1	0
1/128	1/32	23/128	9/16

Table 3: Probability distribution of $c = ab + b'(a + a')$

3 Security Statement and Analysis

This section provides the security statement and analysis of NTRE. Assuming the hardness of the $\mathcal{R}$ -NTRU $_{\eta}$ and $\mathcal{R}$ -LWE $_{\eta}$ problems, the NTRE key encapsulation mechanism we construct is IND-CCA2 secure. Section 3.1 introduces the necessary preliminaries. Section 3.2 presents the theoretical security proofs of NTRE in the Random Oracle Model (ROM) and the Quantum Random Oracle Model (QROM). Section 3.3 discusses the concrete security of the scheme.

3.1 Preliminaries

This section recalls the necessary cryptographic primitives, the relevant notions of the (quantum-accessible) random oracle model, and essential tools for the theoretical security analysis, including the compressed oracle technique, the one-way to hiding lemma, and the difference lemma, thereby laying the groundwork for the security proof of NTRE.

3.1.1 Cryptographic Primitives

Definition 3.1 ($\mathcal{R}$ -NTRU $_{\eta}$ Assumption [11, Definition 4.2]). *Let p be a prime co-prime to q , and let $\mathbf{f} := p\mathbf{f}' + 1$ be invertible in $\mathcal{R}_q$, where the coefficients of $\mathbf{g}, \mathbf{f}' \in \mathcal{R}_q$ follow the distribution η . Set $\mathbf{h} := \mathbf{g}/\mathbf{f} \in \mathcal{R}_q$. The $\mathcal{R}$ -NTRU $_{\eta}$ assumption states that no polynomial-time adversary can distinguish $\mathbf{h}$ from a uniformly random element of $\mathcal{R}_q$ with non-negligible advantage.*

Formally, define the distinguishing advantage of an adversary $\mathcal{A}$ as

$$\text{Adv}_{n,q}^{\mathcal{R}\text{-NTRU}_{\eta}}(\mathcal{A}) := \left| \Pr[\mathcal{A}(\mathbf{h}) = 1 : \mathbf{h} := \mathbf{g}/\mathbf{f}] - \Pr[\mathcal{A}(\mathbf{h}) = 1 : \mathbf{h} \leftarrow \$ \mathcal{R}_q] \right| ,$$

which must be negligible in the security parameter.

Definition 3.2 ($\mathcal{R}$ -LWE $_{\eta}$ Assumption [11, Definition 4.3]). *Let $\mathbf{h} \leftarrow \$ \mathcal{R}_q$, and let $\mathbf{r}, \mathbf{e} \in \mathcal{R}_q$ be polynomials whose coefficients are drawn independently from the distribution η . Set $\mathbf{c} := \mathbf{h} \cdot \mathbf{r} + \mathbf{e} \in \mathcal{R}_q$. The search $\mathcal{R}$ -LWE $_{\eta}$ assumption states that no polynomial-time adversary can recover $\mathbf{e}$ from $(\mathbf{h}, \mathbf{c})$ with non-negligible advantage.*

Formally, define the advantage of an adversary $\mathcal{A}$ as

$$\text{Adv}_{n,q}^{\mathcal{R}\text{-LWE}_{\eta}}(\mathcal{A}) := \Pr[\mathcal{A}(\mathbf{h}, \mathbf{c}) = \mathbf{e}] ,$$

which must be negligible in the security parameter.

Definition 3.3 ($\mathcal{R}$ -LWE2 $_{\eta}$ Assumption [11, Definition 4.5]). *Let $\mathbf{h} \leftarrow \$ \mathcal{R}_q$, and let $\mathbf{r}, \mathbf{e} \in \mathcal{R}_q$ be polynomials whose coefficients are drawn independently from the distribution η . Set $\mathbf{c} := \mathbf{h} \cdot \mathbf{r} + \mathbf{e} \in \mathcal{R}_q$. The $\mathcal{R}$ -LWE2 $_{\eta}$ assumption states that no polynomial-time adversary can recover $\mathbf{e} \bmod 2$ from $(\mathbf{h}, \mathbf{c})$ with non-negligible advantage.*

Formally, define the advantage of an adversary $\mathcal{A}$ as

$$\text{Adv}_{n,q}^{\mathcal{R}\text{-LWE2}_{\eta}}(\mathcal{A}) := \Pr[\mathcal{A}(\mathbf{h}, \mathbf{c}) = \mathbf{e} \bmod 2] ,$$

which must be negligible in the security parameter.

As analyzed in [11], the concrete hardness of the $\mathcal{R}$ -LWE2 $_{\eta}$ problem is equivalent to that of $\mathcal{R}$ -LWE $_{\eta}$.

Definition 3.4 (Public-Key Encryption, PKE). A public-key encryption (PKE) scheme $\text{PKE} = (\text{KGen}, \text{Enc}, \text{Dec})$ is defined over a message space $\mathcal{M}$ and a ciphertext space $\mathcal{C}$, and is parameterized by a security parameter 1^λ . It consists of three probabilistic polynomial-time algorithms:

- *Key generation*: $(\text{pk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda)$ takes as input 1^λ and outputs a public/private key pair (pk, sk) . The public key pk implicitly defines a randomness space $\mathcal{R} := \mathcal{R}(\text{pk})$.
- *Encryption*: $c \leftarrow \text{Enc}(\text{pk}, m)$ takes as input a public key pk and a message $m \in \mathcal{M}$, and outputs a ciphertext $c \in \mathcal{C}$. When we need to make the randomness explicit, we write $c := \text{Enc}(\text{pk}, m; \rho)$ with $\rho \leftarrow \$ \mathcal{R}$.
- *Decryption*: $m := \text{Dec}(\text{sk}, c)$ is deterministic, takes as input a secret key sk and a ciphertext $c \in \mathcal{C}$, and outputs either a message $m \in \mathcal{M}$ or a special symbol $\perp \notin \mathcal{M}$.

The scheme is perfectly correct if for all $(\text{pk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda)$, all $m \in \mathcal{M}$, and all $\rho \leftarrow \$ \mathcal{R}$, we have $\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m; \rho)) = m$.

Definition 3.5 (Worst/Average-Case Correctness of PKE). Let $\text{PKE} = (\text{KGen}, \text{Enc}, \text{Dec})$ be a PKE scheme defined over $(\mathcal{M}, \mathcal{C})$.

- PKE is δ_{wc} -worst-case correct (or has worst-case decryption error rate δ_{wc}) if

$$\mathbb{E}_{(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)} \left[\max_{m \in \mathcal{M}} \Pr_{\rho \leftarrow \$ \mathcal{R}} [\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m; \rho)) \neq m] \right] \leq \delta_{wc}. \quad (10)$$

- PKE is δ_{ac} -average-case correct (or has average-case decryption error rate δ_{ac}) if

$$\mathbb{E}_{(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)} \left[\Pr_{m \leftarrow \$ \mathcal{M}, \rho \leftarrow \$ \mathcal{R}} [\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m; \rho)) \neq m] \right] \leq \delta_{ac}. \quad (11)$$

Definition 3.6 (OW-CPA Security of PKE). A PKE scheme $\text{PKE} = (\text{KGen}, \text{Enc}, \text{Dec})$ defined over $(\mathcal{M}, \mathcal{C})$ is OW-CPA-secure if for any PPT adversary $\mathcal{A}$, the advantage $\text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{A})$ is negligible, where the security notion is defined via the game in Fig. 2 between a challenger and $\mathcal{A}$.

OW-CPA Game:
1 : $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$
2 : $m^* \leftarrow \$ \mathcal{M}$
3 : $c^* \leftarrow \text{Enc}(\text{pk}, m^*)$
4 : $\hat{m} \leftarrow \mathcal{A}(\text{pk}, c^*)$
5 : return $[m^* = \hat{m}]$

Figure 2: OW-CPA security game for $\text{PKE} = (\text{KGen}, \text{Enc}, \text{Dec})$.

The adversary's advantage is defined as

$$\text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{A}) := \Pr[m^* = \hat{m}].$$

The message m^* and ciphertext c^* generated by the challenger are called the challenge message and the challenge ciphertext, respectively.

Definition 3.7 (Key Encapsulation Mechanism). A key encapsulation mechanism $\text{KEM} = (\text{KGen}, \text{Encaps}, \text{Decaps})$ with key space $\mathcal{K}$ consists of the following three algorithms:

- *key generation*: $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$ is a probabilistic algorithm that takes the security parameter 1^λ as input and outputs a public/private key pair (pk, sk) . The public key pk implicitly defines a randomness space $\mathcal{R} := \mathcal{R}(\text{pk})$
- *Encapsulation*: $(c, k) \leftarrow \text{Encaps}(\text{pk})$ is a probabilistic algorithm that takes the public key pk as input, and outputs a ciphertext c and a key $k \in \mathcal{K}$.
- *Decapsulation*: $k := \text{Decaps}(\text{sk}, c)$ is a deterministic algorithm that takes the private key sk and a ciphertext c as input, and outputs a key $k \in \mathcal{K}$ or a special symbol $\perp \notin \mathcal{K}$ indicating that c is not a valid ciphertext.

The scheme is perfect correct if for all $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$ and for all $(k, c) \leftarrow \text{Encaps}(\text{pk})$, we have $\text{Decaps}(\text{sk}, c) = k$. A KEM is said to be with explicit rejection if $\perp \notin \mathcal{K}$, or with implicit rejection if $\perp \in \mathcal{K}$ denotes a random value.

Definition 3.8 (Correctness of KEM). A KEM $\text{KEM} = (\text{KGen}, \text{Encaps}, \text{Decaps})$ is said to have decapsulation error δ if

$$\Pr_{(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)} [\text{Decaps}(\text{sk}, c) \neq k : (k, c) \leftarrow \text{Encaps}(\text{pk})] \leq \delta .$$

Definition 3.9 (IND-CCA2 Security of KEM). Let $\text{KEM} = (\text{KGen}, \text{Encaps}, \text{Decaps})$ be a KEM defined over $(\mathcal{K}, \mathcal{C})$. Its IND-CCA2 security is defined via the game in Fig. 3 between a challenger and an adversary $\mathcal{A}$.

IND-CCA Game:	Decapsulation oracle $O_{\text{Dec}}(c \in \mathcal{C})$:
1 : $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$	1 : if $c = c^*$ then
2 : $(k_0, c^*) \leftarrow \text{Encaps}(\text{pk})$	2 : return $\perp$
3 : $k_1 \xleftarrow{\$} \mathcal{K}, b \xleftarrow{\$} \{0, 1\}$	3 : else
4 : $\hat{b} \leftarrow \mathcal{A}^{O_{\text{Dec}}}(\text{pk}, c^*, k_b)$	4 : $k := \text{Decaps}(\text{sk}, c)$
5 : return $[b = \hat{b}]$	5 : return k

Figure 3: IND-CCA security game for $\text{KEM} = (\text{Gen}, \text{Encaps}, \text{Decaps})$.

The adversary's advantage is defined as

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}(\mathcal{A}) := |\Pr[b = \hat{b}] - 1/2| .$$

If this advantage is negligible for every PPT adversary $\mathcal{A}$, then KEM is said to be IND-CCA2-secure. The keys k_0, k_1 and ciphertext c generated by the challenger are referred to as the challenge keys and challenge ciphertext, respectively. For clarity, we sometimes refer to k_0 as the real key and k_1 as the random key.

3.1.2 The (Quantum-Accessible) Random Oracle Model

We refer the reader to [17] for a standard introduction to quantum computation. In brief, the *state space* of a quantum system is a complex Hilbert space equipped with an inner product. Dirac notation is used to denote state vectors: $|\psi\rangle$ represents a unit vector in the state space, and its dual is denoted $\langle\psi|$. The space admits an orthonormal basis, called the *computational basis*. The joint state of two systems in states $|\psi\rangle$ and $|\phi\rangle$ is given by the tensor product $|\psi\rangle \otimes |\phi\rangle$, often abbreviated as $|\psi\rangle |\phi\rangle$ or $|\psi, \phi\rangle$. The norm of $|\psi\rangle$ is defined as $\| |\psi\rangle \| = \sqrt{\langle\psi|\psi\rangle}$, where $\langle\psi|\phi\rangle$ denotes the inner product of $|\psi\rangle$ and $|\phi\rangle$.

The random oracle model (ROM), introduced by Bellare and Rogaway [4], idealizes a hash function as a publicly accessible random oracle. In this model, an adversary can only obtain the value of a hash function $H(x)$ by querying the oracle at x ; as if H were a truly random function.

The quantum-accessible ROM (QROM) extends the ROM by allowing adversaries to query the random oracle in superposition. A quantum-accessible random oracle (QRO) H is implemented as a unitary operator U_H acting as $|x, y\rangle \mapsto |x, y \oplus H(x)\rangle$. Classical queries remain admissible; we write $y := H(x)$ to denote a classical query to H at x that returns a classical output y . This can be viewed as querying the QRO with the state $|x, 0\rangle$ and then measuring the second register in the computational basis to obtain the classical outcome [10].

3.1.3 The Compressed Oracle Technique

In the classical ROM, reductions can record adversaries' queries. In the QROM, this ability was long considered unattainable due to the quantum no-cloning principle: any direct measurement of a quantum register inevitably disturbs the adversary's state. However, Zhandry [19] circumvented this "recording barrier" by proposing the compressed oracle technique. The core idea is to purify the quantum random oracle and then record query information on the purified quantum random oracle.

Definition 3.10 (Compressed Standard Oracle). *Let D be a database comprising q entries of the form $(x, y) \in (\mathcal{X} \times \mathcal{Y}) \cup \{(\perp, 0^n)\}$, where $n := \log |\mathcal{Y}|$ and q is an upper bound on the number of quantum random oracle queries. The structure of D is*

$$D = ((x_1, y_1), (x_2, y_2), \dots, (x_l, y_l), (\perp, 0^n), \dots, (\perp, 0^n)) ,$$

with $0 \leq l \leq q$, $(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}$, $x_1 < \dots < x_l$, and exactly $q - l$ trailing copies of $(\perp, 0^n)$. We write $\mathcal{D}$ for the set of all such databases. We sometimes explicitly denote a database with q entries by D_q , and let $\mathcal{D}_q$ be the set of such databases. For any $x \in \mathcal{X}$, if there exists a pair (x, y) in D we set $D(x) = y$; otherwise $D(x) = \perp$. By construction, no two distinct entries share the same x . The quantity $|D|$ counts the entries with $x \neq \perp$. When $|D| < q$ and $D(x) = \perp$, the operation $D \cup (x, y)$ removes one $(\perp, 0^n)$ and inserts (x, y) while maintaining the ascending order of the x 's.

Let $\mathbf{D}$ be a register with state space $\mathcal{H} = \mathbb{C}[\mathcal{D}]$. For a basis state $|D\rangle$ ($D \in \mathcal{D}$), we define a unitary decompression operator F_x as follows:

- If $D(x) = \perp$ and $|D| < q$,

$$\begin{aligned} F_x |D\rangle &= 2^{-n/2} \sum_y |D \cup (x, y)\rangle , \quad F_x \left(2^{-n/2} \sum_y |D \cup (x, y)\rangle \right) = |D\rangle , \\ F_x \left(2^{-n/2} \sum_y (-1)^{z \cdot y} |D \cup (x, y)\rangle \right) &= 2^{-n/2} \sum_y (-1)^{z \cdot y} |D \cup (x, y)\rangle \quad (z \neq 0). \end{aligned}$$

- If $D(x) = \perp$ but $|D| = q$, then $F_x |D\rangle = |D\rangle$.

Let $\mathbf{X}$ and $\mathbf{Y}$ be the input and output registers of the quantum random oracle. We define a unitary O_x acting on $\mathbf{YD}$ by

$$O_x : |y, D\rangle_{\mathbf{YD}} \mapsto |y \oplus D(x), D\rangle_{\mathbf{YD}} ,$$

where the XOR operation $\oplus$ is extended to $\perp$ following the convention of [8], i.e., $0^n \oplus \perp = \perp$, $\perp \oplus 0^n = \perp$, $\perp \oplus \perp = 0^n$, and for any $y \neq 0^n$, $y \oplus \perp = y$, $\perp \oplus y = \perp$. Finally, the compressed standard oracle acting on $\mathbf{XYD}$ is defined as

$$\text{CStO} := \sum_x |x\rangle\langle x| \otimes F_x O_x F_x .$$

When the context permits, we refer to the compressed standard oracle simply as the *compressed oracle*. Zhandry [19] proved that the compressed standard oracle is perfectly indistinguishable from the quantum-accessible random oracle.

Lemma 3.1 (Lemma 4 in [19]). *The compressed oracle as defined in Definition 3.10 with $\mathbf{D}$ set as $\bigotimes_{i=1}^q (\perp, 0^n)$ initially is perfectly indistinguishable from a quantum random oracle $H : \mathcal{X} \rightarrow \mathcal{Y}$ for any quantum adversary making at most q random oracle queries.*

3.1.4 The One-Way to Hiding Lemma

Definition 3.11 (Semi-Classical Oracle [3], adapted). *For a subset $S \subseteq \mathcal{X}$, define an indicator function $f : \mathcal{X} \rightarrow \{0, 1\}$ with $f(x) = 1$ iff $x \in S$. The semi-classical oracle $\mathcal{O}_f^{\text{SC}}$ is a quantum-accessible oracle that, on a query state $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle$, performs the following:*

1. *Initialize a single-qubit register $\mathbf{B}$ in $|0\rangle_{\mathbf{B}}$.*
2. *Transform $\sum_x \alpha_x |x\rangle |0\rangle_{\mathbf{B}} \mapsto \sum_x \alpha_x |x\rangle |f_S(x)\rangle_{\mathbf{B}}$ and immediately measure $\mathbf{B}$ in the computational basis.*

*The measurement outcome is output by the oracle. Sometimes we also denote this semi-classical oracle by $\mathcal{O}_S^{\text{SC}}$. During the execution of a quantum oracle algorithm $A^{|\mathcal{O}_S^{\text{SC}}\rangle}$, we denote by **Find** the classical event that the measurement of $\mathcal{O}_S^{\text{SC}}$ ever yields outcome 1. It is not hard to see that once the event **Find** occurs, immediately measuring the input register yields a value $x \in S$.*

Lemma 3.2 (Semi-Classical One-Way to Hiding [3, Theorem 1], adapted). *Let $S \subseteq \mathcal{X}$ be a random set and let $G, H : \mathcal{X} \rightarrow \mathcal{Y}$ be random functions such that $G(x) = H(x)$ for all $x \notin S$. Let z be a random bitstring. The tuple (G, H, S, z) may follow an arbitrary joint distribution $\mathfrak{D}$. Denote by $H \setminus S$ a punctured oracle that first queries $\mathcal{O}_S^{\text{SC}}$ on the input register of H and then queries H itself. Let A be a quantum oracle algorithm that makes at most q queries. Define*

$$\begin{aligned} P_{\text{left}} &:= \Pr[1 \leftarrow A^{|\mathcal{H}\rangle}(z) : (G, H, S, z) \leftarrow \mathfrak{D}] , \\ P_{\text{right}} &:= \Pr[1 \leftarrow A^{|\mathcal{G}\rangle}(z) : (G, H, S, z) \leftarrow \mathfrak{D}] , \\ P_{\text{find}} &:= \Pr[\text{Find} : A^{|\mathcal{G} \setminus S\rangle}(z), (G, H, S, z) \leftarrow \mathfrak{D}] . \end{aligned}$$

Then we have

$$|P_{\text{left}} - P_{\text{right}}| \leq 2\sqrt{(q+1) \cdot P_{\text{find}}} , \quad \left| \sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}} \right| \leq 2\sqrt{(q+1) \cdot P_{\text{find}}} .$$

Lemma 3.3 (Search in a Semi-Classical Oracle [3, Theorem 2], adapted). *Let $S \subseteq \mathcal{X}$ be a random set and let z be a random bitstring. The pair (S, z) may follow an arbitrary joint distribution $\mathfrak{D}$. Let A be any quantum oracle algorithm that makes at most q queries to a semi-classical oracle with domain $\mathcal{X}$. Define an algorithm B which, on input z , proceeds as follows: it picks $i^* \leftarrow \$ \{1, \dots, q\}$ uniformly at random, runs $A^{|\mathcal{O}_0^{SC}\rangle}(z)$ until (just before) its i^* -th query, measures the input register of that query in the computational basis, and outputs the measured value x . Then*

$$\Pr[\text{Find} : A^{|\mathcal{O}_0^{SC}\rangle}(z), (S, z) \leftarrow \mathfrak{D}] \leq 4q \cdot \Pr[x \in S : x \leftarrow B(z), (S, z) \leftarrow \mathfrak{D}] .$$

If S and z are independent, define $P_{\max} := \max_{x \in \mathcal{X}} \Pr[x \in S]$. Then we have

$$\Pr[x \in S : x \leftarrow B(z), (S, z) \leftarrow \mathfrak{D}] \leq P_{\max} .$$

Definition 3.12 (Compressed Semi-Classical Oracle [12], adapted). *Let $\mathcal{D}_q$ be a set of databases and $S \subseteq \mathcal{D}_q$. Define a function $f_S : \mathcal{D}_q \rightarrow \{0, 1\}$ such that $f_S(D) = 1$ if $D \in S$ and $f_S(D) = 0$ otherwise. The compressed semi-classical oracle $\mathcal{O}_S^{CSC}$ operates on an input state $\sum_{z \in \{0,1\}^*, D \in \mathcal{D}_q} \alpha_{z,D} |z, D\rangle$ as follows:*

1. Initialize a single qubit register L as $|0\rangle_{\mathsf{L}}$.
2. Transform $\sum_{z,D} \alpha_{z,D} |z, D\rangle |0\rangle_{\mathsf{L}} \mapsto \sum_{z,D} \alpha_{z,D} |z, D\rangle |f_S(D)\rangle_{\mathsf{L}}$ and immediately measure L in the computational basis.

The measurement outcome is output by the oracle. During the execution of a quantum oracle algorithm $A^{|\mathcal{O}_S^{CSC}\rangle}$, we denote by **Find** the classical event that the measurement of $\mathcal{O}_S^{CSC}$ (on L) ever has outcome 1.

Lemma 3.4 (Compressed Semi-Classical One-Way to Hiding [12, Theorem 1]). *Let $\mathcal{D}_q$ be the database register defined over $\mathcal{D}_q$ and let $\mathsf{H}_c : \{0, 1\}^m \rightarrow \{0, 1\}^n$ be the compressed standard oracle implemented on $\mathcal{D}_q$. Let $S \subseteq \mathcal{D}_q$ be a subset such that $D_{\perp} \notin S$, where $D_{\perp}$ denotes the database containing only q pairs $(\perp, 0^n)$, and let z be a random string. The pair (S, z) may follow an arbitrary joint distribution $\mathfrak{D}$. Define $\mathsf{H}_c \setminus S$ as an oracle that first queries H_c and then queries $\mathcal{O}_S^{CSC}$ on the database register $\mathcal{D}_q$. Let A be a quantum oracle algorithm (not necessarily unitary) that has oracle access to H_c and oRead_f , and suppose A makes at most $q_H \leq q$ queries to H_c and q_R queries to oRead_f , where oRead_f (with f possibly depending on z) is simulated by the database read operation Read_f defined as $\text{Read}_f : |x, D, y\rangle_{\mathcal{D}_q \mathsf{Y}} \mapsto |x, D, y \oplus f(x, D)\rangle_{\mathcal{D}_q \mathsf{Y}}$. Define*

$$\begin{aligned} P_{\text{left}} &:= \Pr[1 \leftarrow A^{|\mathsf{H}_c\rangle, |\text{oRead}_f\rangle}(z) : (S, z) \leftarrow \mathfrak{D}] , \\ P_{\text{right}} &:= \Pr[1 \leftarrow A^{|\mathsf{H}_c \setminus S\rangle, |\text{oRead}_f\rangle}(z) : (S, z) \leftarrow \mathfrak{D}] , \\ P_{\text{find}} &:= \Pr[\text{Find} : A^{|\mathsf{H}_c \setminus S\rangle, |\text{oRead}_f\rangle}(z), (S, z) \leftarrow \mathfrak{D}] . \end{aligned}$$

Then we have

$$|P_{\text{left}} - P_{\text{right}}| \leq \sqrt{(q_H + 1) \cdot P_{\text{find}}} , \quad \left| \sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}} \right| \leq \sqrt{(q_H + 1) \cdot P_{\text{find}}} .$$

Define a projector on the database register $\mathcal{D}_q$ as $J_S := \sum_{D \in S} |D\rangle\langle D|$. Then the probability P_{find} defined above and the compressed standard oracle CStO given in Definition 3.10 satisfy

$$P_{\text{find}} \leq q_H \cdot \mathbb{E}_{(S,z) \leftarrow \mathfrak{D}} [\| [J_S, \text{CStO}] \|^2] ,$$

where $[A, B] := AB - BA$ denotes the commutator between A and B [17, (2.66)].

3.1.5 Difference Lemma

Lemma 3.5 (Difference Lemma [5, Theorem 4.7]). *Let W_1 , W_2 , and Z be events defined over some probability space, and let $\bar{Z}$ denote the complement of Z . If $\Pr[W_1 \wedge \bar{Z}] = \Pr[W_2 \wedge \bar{Z}]$, then*

$$|\Pr[W_2] - \Pr[W_1]| \leq \Pr[Z] .$$

3.2 Theoretical Security

We first prove the OW-CPA security of NTRE.CPAPKE in the standard model by reducing it to the $\mathcal{R}$ -NTRU $_{\psi_1}$ and $\mathcal{R}$ -LWE2 $_{\psi_1}$ assumptions. Then, in the classical random oracle model (ROM), we establish the IND-CCA2 security of NTRE.CCAKEM through a sequence of game hops, and relate its security to the OW-CPA security of the underlying NTRE.CPAPKE. Subsequently, we extend the proof to the quantum-accessible random oracle model (QROM) and obtain a complete security reduction for NTRE.CCAKEM in the quantum setting.

3.2.1 Security of NTRE.CPAPKE

The algorithmic structure of NTRE.CPAPKE, as presented in Section 1.3.1, can be reduced to the scheme $\text{PKE}' = (\text{KGen}', \text{Enc}', \text{Dec}')$ shown in Figure 4, which is defined over $\mathcal{R}_q$ with distribution ψ_1^n . Thus, the OW-CPA security of NTRE.CPAPKE follows from that of PKE' .

$\text{KGen}'(1^\lambda)$	$\text{Enc}'(\text{pk} = \mathbf{h}, m \in \{0, 1\}^n)$
1 : repeat	1 : $\mathbf{r} \leftarrow \psi_1^n$
2 : $\mathbf{f}' \leftarrow \psi_1^n$	2 : $r' \leftarrow \{0, 1\}^n$
3 : $\mathbf{f} := 2 \cdot \mathbf{f}' + 1$	3 : $\mathbf{e} := \text{CBD}'_1(m, r')$
4 : until $\mathbf{f}$ is invertible $\mathcal{R}_q$	4 : return $\mathbf{c} := \mathbf{h} \cdot \mathbf{r} + \mathbf{e}$
5 : $\mathbf{g}' \leftarrow \psi_1^n$	$\text{Dec}'(\text{sk} = \mathbf{f}, \mathbf{c} \in \mathcal{R}_q)$
6 : $\mathbf{g} := 2 \cdot \mathbf{g}'$	1 : $\mathbf{m} := (\mathbf{c} \cdot \mathbf{f} \bmod^\pm q) \bmod 2$
7 : $\mathbf{h} := \mathbf{g} \cdot \mathbf{f}^{-1}$	2 : $m := \mathbf{m} \ (\in \{0, 1\}^n)$
8 : return $(\text{pk}, \text{sk}) := (\mathbf{h}, \mathbf{f})$	3 : return m

Figure 4: $\text{PKE}' = (\text{KGen}', \text{Enc}', \text{Dec}')$, where ψ_1^n denotes the distribution over $\mathcal{R}_q$ with each coefficient drawn independently from ψ_1 .

As stated in Theorem 3.1, the OW-CPA security of PKE' is based on the $\mathcal{R}$ -NTRU $_\eta$ and $\mathcal{R}$ -LWE2 $_\eta$ assumptions with $\eta = \psi_1$.

Theorem 3.1 (OW-CPA Security of PKE'). *Let ψ_1 be the centered binomial distribution on $\{-1, 0, 1\}$. In the PKE scheme PKE' shown in Figure 4, the coefficients of $\mathbf{f}', \mathbf{g}, \mathbf{r} \in \mathcal{R}_q$ are drawn from ψ_1^n , and the plaintext is $m \in \{0, 1\}^n$. Then for any PPT adversary $\mathcal{A}$ attacking the OW-CPA security of PKE' as defined in Definition 3.6, there exist a PPT adversary $\mathcal{B}$ against the $\mathcal{R}$ -NTRU $_{\psi_1}$ assumption and a PPT adversary $\mathcal{C}$ against the $\mathcal{R}$ -LWE2 $_{\psi_1}$ assumption such that*

$$\text{Adv}_{\text{PKE}'}^{\text{OW-CPA}}(\mathcal{A}) \leq \text{Adv}_{n,q}^{\mathcal{R}\text{-NTRU}_{\psi_1}}(\mathcal{B}) + \text{Adv}_{n,q}^{\mathcal{R}\text{-LWE2}_{\psi_1}}(\mathcal{C}) .$$

Proof (Theorem 3.1). We prove Theorem 3.1 by defining two games G_0 and G_1 , played between a challenger and a PPT adversary $\mathcal{A}$, where $\Pr[G_i^{\mathcal{A}} \Rightarrow 1]$ denotes the probability that Game G_i outputs 1.

Game G_0 . This is the original OW-CPA security game for PKE' as defined in Definition 3.6. Thus, the advantage of $\mathcal{A}$ is

$$\text{Adv}_{\text{PKE}'}^{\text{OW-CPA}}(\mathcal{A}) = \Pr[G_0^{\mathcal{A}} \Rightarrow 1] .$$

Game G_1 . In Game G_1 , the public key $\mathbf{h}$ is replaced by a uniformly random polynomial $\mathbf{h} \leftarrow \mathcal{R}_q$. Everything else remains exactly as in G_0 . Distinguishing G_0 from G_1 is precisely the $\mathcal{R}\text{-NTRU}_{\psi_1}$ problem, hence there exists a PPT adversary $\mathcal{B}$ such that

$$|\Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \Pr[G_1^{\mathcal{A}} \Rightarrow 1]| \leq \text{Adv}_{n,q}^{\mathcal{R}\text{-NTRU}_{\psi_1}}(\mathcal{B}) .$$

Because $\mathbf{h}$ is now uniformly random, the challenge ciphertext $\mathbf{c} = \mathbf{h} \cdot \mathbf{r} + \mathbf{e}$ with $\mathbf{e} = \text{CBD}'_1(m, r')$ forms an instance of the $\mathcal{R}\text{-LWE2}_{\psi_1}$ problem: recovering m from $(\mathbf{h}, \mathbf{c})$ is equivalent to solving $\mathcal{R}\text{-LWE2}_{\psi_1}$. Thus, there exists a PPT adversary $\mathcal{C}$ for which

$$\Pr[G_1^{\mathcal{A}} \Rightarrow 1] = \text{Adv}_{n,q}^{\mathcal{R}\text{-LWE2}_{\psi_1}}(\mathcal{C}) .$$

Combining the two bounds, we obtain

$$\text{Adv}_{\text{PKE}'}^{\text{OW-CPA}}(\mathcal{A}) \leq \text{Adv}_{n,q}^{\mathcal{R}\text{-NTRU}_{\psi_1}}(\mathcal{B}) + \text{Adv}_{n,q}^{\mathcal{R}\text{-LWE2}_{\psi_1}}(\mathcal{C}) ,$$

which completes the proof. $\square$

3.2.2 ROM Security of NTRE.CCAKEM

Starting from this section, we analyze the security of NTRE.CCAKEM. In this section we first consider the classical random oracle model (ROM). As described in Section 2.1, NTRE.CCAKEM is obtained from NTRE.CPAPKE via the FOAC' transform of Figure 1, where $\mathcal{M}_1 = \mathcal{B}^{\lfloor n/16 \rfloor}$ and $\mathcal{M}_2 = \mathcal{B}^{\lceil n/16 \rceil}$. Hence, its IND-CCA2 security follows from that of $\text{KEM} := \text{FOAC}'[\text{PKE}', F, H]$, whose security is captured by Theorem 3.2.

Theorem 3.2 (ROM Security of KEM). *Let $F : \mathcal{M}_1 \rightarrow \mathcal{M}_2$ and $H : \mathcal{M}_2 \rightarrow \mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}$ be modeled as random oracles. If the underlying PKE scheme PKE' is δ_{ac} -average-case correct, then for any PPT adversary $\mathcal{A}$ against the IND-CCA2 security of the KEM $\text{KEM} := \text{FOAC}'[\text{PKE}', F, H]$, there exists a PPT adversary $\mathcal{B}$ against the OW-CPA security of PKE' such that*

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}(\mathcal{A}) \leq q_F q_H \text{Adv}_{\text{PKE}'}^{\text{OW-CPA}}(\mathcal{B}) + (q_H + 1)\delta_{ac} + (q_D + q_H)/|\mathcal{M}_1| + \Delta ,$$

where q_F, q_H, q_D bound the number of queries made by $\mathcal{A}$ to the random oracles F, H and the decapsulation oracle, respectively, and $\Delta \leq (q_D + 1)(q_H + 1)/|\mathcal{M}_2|$.

Proof Sketch. The proof of Theorem 3.2 consists of three main steps. First, we simulate the decapsulation oracle without using the secret key $\mathbf{sk}$ by retrieving answers from the random oracle query history. Next, we remove from the history all entries that were inserted by the challenger during the generation of the challenge ciphertext and the challenge key. We then reprogram the oracle to decouple the challenge, and finally reduce the adversary's advantage to the CPA security of the underlying PKE, where the reduction relies on the adversary's ability to detect the reprogrammed point.

Games G_0 to G_6 :	16 : if $r \notin \text{Domain}(Map_2)$ then
1 : $(pk, sk) \leftarrow \text{Gen}(1^\lambda), r^* \leftarrow \$ \mathcal{M}_2$	$Map_2[r] \leftarrow \$ \mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}$
2 : initialize empty associative arrays	17 : return $Map_2[r]$
$Map_1 : \mathcal{M}_1 \rightarrow \mathcal{M}_2$	Decapsulation Oracle $O_{\text{Dec}}(c)$:
$Map_2 : \mathcal{M}_2 \rightarrow \mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}$	18 : if $c = c^*$ then
3 : $(m_1^*, \rho^*, k_0) \leftarrow \$ \mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}$	return $\perp$
4 : $u^* \leftarrow \$ \mathcal{M}_2$	19 : $m_1 \ m_2 := \text{Dec}'(sk, c) \quad // G_0-G_2$
5 : $Map_2[r^*] := (m_1^*, \rho^*, k_0) \quad // G_0-G_3$	20 : if $m_1 \ m_2 = \perp$ then
6 : $Map_1[m_1^*] := u^* \quad // G_0-G_5$	return $\perp \quad // G_0-G_2$
7 : $m_2^* := r^* \oplus u^*$	21 : if $m_1 \notin \text{Domain}(Map_1)$
8 : $c^* := \text{Enc}'(pk, m_1^* \ m_2^*; \rho^*) \quad // G_0-G_6$	return $\perp \quad // G_2$
9 : $b \leftarrow \$ \{0, 1\}, k_1 \leftarrow \$ \mathcal{K}$	22 : $r := m_2 \oplus F(m_1) \quad // G_0-G_2$
10 : $\hat{b} \leftarrow \mathcal{A}^{F, H, O_{\text{Dec}}}(pk, c^*, k_b)$	23 : if $r \notin \text{Domain}(Map_2)$
11 : return $[b = \hat{b}]$	return $\perp \quad // G_1-G_2$
Random Oracle $F(m_1)$:	24 : $(m'_1, \rho, k) := H(r) \quad // G_0-G_2$
12 : if $m_1 = m_1^*$ then	25 : if $m_1 = m'_1$ and
$Map_1[m_1^*] := u^* \quad // G_6$	$\text{Enc}'(pk, m'_1 \ m_2; \rho) = c$ then
13 : if $m_1 \notin \text{Domain}(Map_1)$ then	return $k \quad // G_0-G_2$
$Map_1[m_1] \leftarrow \$ \mathcal{M}_2$	26 : if $\exists (r, (m_1, \rho, k)) \in Map_2$ and
14 : return $Map_1[m_1]$	$(m_1, u) \in Map_1$ s.t.
Random Oracle $H(r)$:	$\text{Enc}'(pk, m_1 \ (r \oplus u); \rho) = c$ then
15 : if $r = r^*$ then	return $k \quad // G_3-G_6$
return $(m_1^*, \rho^*, k_0) \quad // G_4$	27 : return $\perp$

Figure 5: Games G_0 to G_6 for the proof of Theorem 3.2.

Proof (Theorem 3.2). We define a sequence of games G_j (for $j = 0, \dots, 6$) played between a challenger and an adversary $\mathcal{A}$, as shown in Fig. 5. In each game, b denotes the random bit sampled by the challenger, and $\hat{b}$ is the bit output by $\mathcal{A}$. We use W_j to denote the event that $b = \hat{b}$ in game G_j .

Game G_0 . The challenger's behavior in this game is depicted in Fig. 5. It initializes two associative arrays, $Map_1 : \mathcal{M}_1 \rightarrow \mathcal{M}_2$ and $Map_2 : \mathcal{M}_2 \rightarrow \mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}$, to simulate the random oracles F and H , respectively. The challenger first randomly samples $(m_1^*, \rho^*, k_0) \leftarrow \$ \mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}$ and $u^* \leftarrow \$ \mathcal{M}_2$, and immediately stores them in $Map_2[r^*]$ and $Map_1[m_1^*]$, respectively. This is equivalent to setting $H(r^*) := (m_1^*, \rho^*, k_0)$ and $F(m_1^*) := u^*$. From $\mathcal{A}$'s view, the challenger's behavior is identical to that in the IND-CCA2 game for $\text{KEM} := \text{FOAC}'[\text{PKE}', F, H]$. Therefore, we have

$$|\Pr[W_0] - 1/2| = \text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}(\mathcal{A}).$$

Game G_1 . This game modifies the decapsulation oracle as follows. For a decapsulation oracle query $c \neq c^*$, after computing $m_1 \| m_2 := \text{Dec}'(sk, c) (\neq \perp)$ and $r := m_2 \oplus F(m_1)$, if $r \notin \text{Domain}(Map_2)$, the oracle directly returns $\perp$ (as shown in line 23 of Fig. 5). Let Z_1 be the event that $r \notin \text{Domain}(Map_2)$ yet $m'_1 = m_1$, where $(m'_1, \rho, k) := H(r)$. According

to the simulation of the random oracle H , since m'_1 is sampled uniformly at random from $\mathcal{M}_1$, we have $\Pr[Z_1] \leq 1/|\mathcal{M}_1|$. If Z_1 does not occur, then Games G_1 and G_0 proceed identically. By the Difference Lemma and a union bound, we obtain

$$|\Pr[W_1] - \Pr[W_0]| \leq q_D \cdot \Pr[Z_1] \leq q_D/|\mathcal{M}_1| ,$$

where q_D is the bounded number of decapsulation oracle queries made by $\mathcal{A}$.

Game G_2 . This game continues the modification of the decapsulation oracle. For a decapsulation oracle query $c \neq c^*$, after computing $m_1 \| m_2 := \text{Dec}'(\text{sk}, c)$ ($\neq \perp$), if $m_1 \notin \text{Domain}(\text{Map}_1)$, the oracle directly outputs $\perp$ (as shown in line 21 of Fig. 5). Let Z_2 be the event that $m_1 \notin \text{Domain}(\text{Map}_1)$ but $r := m_2 \oplus F(m_1) \in \text{Domain}(\text{Map}_2)$. Clearly, if event Z_2 does not occur, the challenger's behavior in Game G_2 is identical to that in Game G_1 . According to the simulation of the random oracle F , when $m_1 \notin \text{Domain}(\text{Map}_1)$, the value $F(m_1)$ is uniformly random over $\mathcal{M}_2$. Therefore, for a fixed m_2 , the probability that $m_2 \oplus F(m_1)$ lies in $\text{Domain}(\text{Map}_2)$ is at most $(q_H + 1)/|\mathcal{M}_2|$. Here, q_H is the bounded number of queries $\mathcal{A}$ makes to the random oracle H , and the extra $+1$ in the numerator comes from the entry $(r^*, (m^*, \rho^*, k_0))$ stored in Map_2 during the challenger's initialization, which ensures $|\text{Domain}(\text{Map}_2)| \leq q_H + 1$. Consequently, by the Difference Lemma and a union bound, we have

$$|\Pr[W_2] - \Pr[W_1]| \leq q_D \cdot \Pr[Z_2] \leq q_D(q_H + 1)/|\mathcal{M}_2| .$$

Game G_3 . From this game onward, the decapsulation oracle no longer uses the computation $m_1 \| m_2 := \text{Dec}'(\text{sk}, c)$ to obtain its response. Specifically, lines 19–25 of Fig. 5 are removed and replaced by a search through Map_2 and Map_1 for entries $(r, (m_1, \rho, k))$ and (m_1, u) satisfying $\text{Enc}'(\text{pk}, m_1 \| (r \oplus u); \rho) = c$ (as indicated in line 26 of Fig. 5): if such a pair of entries is found, the oracle returns k ; otherwise, it returns $\perp$. It is clear that for the same decapsulation oracle query $c \neq c^*$, if the oracle in Game G_3 returns $\perp$, then the oracle in Game G_2 must also return $\perp$. This is because if the oracle in Game G_2 did not return $\perp$, it implies that Map_2 and Map_1 must contain entries $(r, (m_1, \rho, k))$ and (m_1, u) satisfying $\text{Enc}'(\text{pk}, m_1 \| (r \oplus u); \rho) = c$, which would in turn prevent the oracle in Game G_3 from returning $\perp$.

Therefore, a difference in the behavior of the decapsulation oracles for a query $c \neq c^*$ between Games G_3 and G_2 can only occur when the oracle in Game G_3 returns some $k \neq \perp$, while the oracle in Game G_2 does not return this k . This implies that the value $m_1 \| (r \oplus u)$ found in Game G_3 is not equal to $\text{Dec}'(\text{sk}, c)$. In other words, there exists an entry $(r, (m_1, \rho, k)) \in \text{Map}_2$ (with a corresponding $(m_1, u) \in \text{Map}_1$) such that $\text{Dec}'(\text{sk}, \text{Enc}'(\text{pk}, m_1 \| (r \oplus u); \rho)) \neq m_1 \| (r \oplus u)$. We denote this event by Z_3 .

Given (pk, sk) , define $\delta(\text{pk}, \text{sk}) = \Pr[\text{Dec}'(\text{sk}, \text{Enc}'(\text{pk}, m_1 \| m_2; \rho)) \neq m_1 \| m_2]$, where the probability is taken over $m_1 \| m_2 \leftarrow \$ \mathcal{M}_1 \times \mathcal{M}_2$ and $\rho \leftarrow \$ \mathcal{R}$. For any fixed $r \in \mathcal{M}_2$, the random variable $r \oplus u$, where $u \leftarrow \$ \mathcal{M}_2$, is uniformly distributed over $\mathcal{M}_2$. Thus, we have $\Pr[Z_3 : (\text{pk}, \text{sk})] \leq |\text{Domain}(\text{Map}_2)| \cdot \delta(\text{pk}, \text{sk}) = (q_H + 1) \delta(\text{pk}, \text{sk})$. By the Difference Lemma and a union bound, it follows that $|\Pr[W_3 : (\text{pk}, \text{sk})] - \Pr[W_2 : (\text{pk}, \text{sk})]| \leq \Pr[Z_3 : (\text{pk}, \text{sk})]$. By averaging over $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$, we can obtain

$$|\Pr[W_3] - \Pr[W_2]| \leq \mathbb{E}[\Pr[Z_3 : (\text{pk}, \text{sk})]] \leq \mathbb{E}[(q_H + 1)\delta(\text{pk}, \text{sk})] \leq (q_H + 1)\delta_{ac} ,$$

where the last inequality follows from the definition of the δ_{ac} -average-case correctness of PKE' (see (11)), i.e., $\mathbb{E}[\delta(\text{pk}, \text{sk})] \leq \delta_{ac}$, under the distribution $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$.

Game G_4 . We now begin modifying the random oracles. In this game, the challenger no longer stores $(r^*, (m_1^*, \rho^*, k_0))$ in Map_2 during initialization (i.e., line 5 of Fig. 5 is

removed), but still uses (m_1^*, ρ^*, k_0) as the value of $H(r^*)$ (as shown in line 15 of Fig. 5). Consequently, in the simulation of the decapsulation oracle in Game G_4 , line 26 will not use $(r^*, (m_1^*, \rho^*, k_0))$ to compute responses to decapsulation oracle queries. It is important to note, however, that line 26 in Game G_3 would not have used $(r^*, (m_1^*, \rho^*, k_0))$ for such computations either. If it did, this would imply that for some decapsulation oracle query $c \neq c^*$, there exist entries $(r^*, (m_1^*, \rho^*, k_0)) \in \text{Map}_2$ and $(m_1^*, u^*) \in \text{Map}_1$ such that $\text{Enc}'(\text{pk}, m_1^* \parallel (r^* \oplus u^*); \rho^*) = c$, which contradicts $c \neq c^*$. Therefore, from $\mathcal{A}$'s perspective, the challenger's behavior in Game G_4 is identical to that in Game G_3 . Hence, we have

$$\Pr[W_4] = \Pr[W_3] .$$

Game G_5 . In this game, the challenger no longer uses (m_1^*, ρ^*, k_0) as the value of $H(r^*)$, i.e., line 15 of Fig. 5 is removed. Let X_j be the event that $\mathcal{A}$ queries r^* to the random oracle H in Game G_j . Clearly, if X_j does not occur, then Games G_5 and G_4 proceed identically. Hence, by the Difference Lemma, we have

$$|\Pr[W_5] - \Pr[W_4]| \leq \Pr[X_5] .$$

Furthermore, note that both k_0 and k_1 , which are uniformly and independently sampled from $\mathcal{K}$, are not used elsewhere in the game at this point. Therefore, the bit b is independent and uniformly random from $\mathcal{A}$'s view. Consequently, we have

$$\Pr[W_5] = 1/2 .$$

Game G_6 . We now analyze X_j . In the initialization phase, the challenger no longer stores (m_1^*, u^*) in Map_1 (i.e., line 6 of Fig. 5 is removed). However, when $\mathcal{A}$ queries $F(m_1^*)$, the random oracle F still returns u^* as the value of $F(m_1^*)$ (as in line 12 of Fig. 5). Note that, in Game G_6 , although $(m_1^*, u^*) \notin \text{Map}_1$ at the beginning of the game, once $\mathcal{A}$ queries $F(m_1^*)$, the pair (m_1^*, u^*) enters Map_1 . This modification causes line 26 in the decapsulation oracle of Game G_6 not to use (m_1^*, u^*) for response computation before $\mathcal{A}$ queries $F(m_1^*)$. In Game G_5 , however, line 26 might still use (m_1^*, u^*) when there exists an entry $(r, (m_1 = m_1^*, \rho, k)) \in \text{Map}_2$. Note that for all $(r, (m_1, \rho, k)) \in \text{Map}_2$, the value m_1 is uniformly random over $\mathcal{M}_1$. Let q_H bound the number of queries $\mathcal{A}$ makes to the random oracle H . The probability that there exists an entry $(r, (m_1 = m_1^*, \rho, k)) \in \text{Map}_2$ is at most $q_H/|\mathcal{M}_1|$. Therefore,

$$|\Pr[X_6] - \Pr[X_5]| \leq q_H/|\mathcal{M}_1| .$$

Recall that X_6 is the event in Game G_6 where $\mathcal{A}$ queries r^* to the random oracle H , with $r^* = m_2^* \oplus u^* = m_2^* \oplus F(m_1^*)$. Importantly, u^* is used only as the value of $F(m_1^*)$, and r^* is not used elsewhere in the game. Thus, if $\mathcal{A}$ has not queried m_1^* to F , then u^* is independent of $\mathcal{A}$'s view, and consequently r^* is also independent of $\mathcal{A}$'s view. Let Y_j be the event that $\mathcal{A}$ queries m_1^* to F in Game G_j . This implies $\Pr[X_6 \wedge \overline{Y_6}] \leq q_H/|\mathcal{M}_2|$, and so

$$\Pr[X_6] = \Pr[X_6 \wedge Y_6] + \Pr[X_6 \wedge \overline{Y_6}] \leq \Pr[X_6 \wedge Y_6] + q_H/|\mathcal{M}_2| .$$

When the event $X_6 \wedge Y_6$ occurs, it means $\mathcal{A}$ has queried m_1^* to F and r^* to H , where $r^* = m_2^* \oplus F(m_1^*)$. In this case, we can construct an adversary $\mathcal{B}$ against the OW-CPA security of the underlying PKE'. Specifically, given (pk, c^*) from the OW-CPA challenger, $\mathcal{B}$ samples $k \leftarrow \mathcal{K}$, sends (pk, c^*, k) to $\mathcal{A}$, and then simulates the random oracle H and the decapsulation oracle for $\mathcal{A}$ exactly as in Game G_6 . For the random oracle F , $\mathcal{B}$ simulates

it as in Game G_6 but with line 12 removed. Note that here $\mathcal{B}$ implicitly chooses a random u^* ; when $\mathcal{A}$ queries $F(m_1^*)$, the oracle responds with this u^* . From $\mathcal{A}$'s view, $\mathcal{B}$'s behavior is identical to the challenger in Game G_6 . At the end of the game, if $X_6 \wedge Y_6$ occurs, $\mathcal{B}$ first randomly selects an m_1 from $\text{Domain}(\text{Map}_1)$ and an r from $\text{Domain}(\text{Map}_2)$, and then outputs $m_1 \parallel (r \oplus F(m_1))$ as its solution to the OW-CPA challenge (pk, c^*) . Since $|\text{Domain}(\text{Map}_1)| \leq q_F$ and $|\text{Domain}(\text{Map}_2)| \leq q_H$, we have

$$\Pr[X_6 \wedge Y_6] / q_F q_H \leq \text{Adv}_{\text{PKE}'}^{\text{OW-CPA}}(\mathcal{B}) .$$

Combining all bounds, we have

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}(\mathcal{A}) \leq q_F q_H \text{Adv}_{\text{PKE}'}^{\text{OW-CPA}}(\mathcal{B}) + (q_H + 1)\delta_{ac} + (q_D + q_H)/|\mathcal{M}_1| + \Delta ,$$

where $\Delta \leq (q_D + 1)(q_H + 1)/|\mathcal{M}_2|$. □

3.2.3 Auxiliary Tools for the QROM Proof

In what follows we analyze the theoretical security of NTRE.CCAKEM in the quantum setting by giving a proof in the quantum-accessible random oracle model (QROM). A central obstacle is that the ROM proof framework for FOAC' does not directly carry over to the quantum setting: one needs a mechanism that records and manipulates superposition queries made by the adversary to the random oracle.

The compressed oracle technique [19] provides exactly such a mechanism. Building on this technique, we extract three reusable core lemmas that will serve as the auxiliary tools for the QROM proof of NTRE.CCAKEM given in the next section. Roughly speaking, the *switching* and *lookup* lemmas allow the reduction to *switch* from calling the quantum random oracle during decapsulation to *checking* entries of the compressed oracle database, while the *puncturing* lemma *removes* database entries that do not correspond to queries made by the adversary.

Each lemma directly mirrors a key step of the corresponding ROM argument and is specifically tailored to the quantum setting. Their roles will become clear in the detailed security analysis of FOAC' presented in the next section. Full proofs of the three lemmas are given in [7].

The Switching Lemma. It allows a simulator (e.g., a decapsulation oracle) to replace a quantum-accessible random oracle (QRO) query $\mathbf{H}(x)$ with a direct database lookup $D(x)$, thereby avoiding extra QRO queries. The replacement is undetectable whenever the fraction of codomain elements of $\mathbf{H}$ that satisfy a specific condition is negligible.

Lemma 3.6 (Switching Lemma). *Let $\mathbf{H} : \mathcal{X} \rightarrow \mathcal{Y}$ be a quantum-accessible random oracle, and let $\mathbf{O}_0$ be a quantum oracle acting on the input registers $\mathbf{WX}$ (defined over $\mathcal{W} \times \mathcal{X}$) and an output register $\bar{\mathbf{Y}}$ (defined over $\bar{\mathcal{Y}}$) as*

$$\mathbf{O}_0 : |w, x, \bar{y}\rangle_{\mathbf{WX}\bar{\mathbf{Y}}} \mapsto |w, x, \bar{y} \oplus f(\mathbf{H}(x))\rangle_{\mathbf{WX}\bar{\mathbf{Y}}} \text{ with } f(y) = \begin{cases} g_{w,x}(y), & \text{if } y \in S_{w,x} \\ g_{w,x}(\perp), & \text{otherwise} \end{cases},$$

where both the function $g_{w,x} : \mathcal{Y} \cup \{\perp\} \rightarrow \bar{\mathcal{Y}}$ and the set $S_{w,x} \subseteq \mathcal{Y}$ are determined by (w, x) . Let $\mathbf{H}_c : \mathcal{X} \rightarrow \mathcal{Y}$ be a compressed standard oracle with a database register $\mathbf{D}$ initialized to $\bigotimes_{i=1}^{q_H} |(\perp, 0^{\log|\mathcal{Y}|})\rangle$, and let $\mathbf{O}_1$ be the quantum oracle

$$\mathbf{O}_1 : |w, x, D, \bar{y}\rangle_{\mathbf{WXD}\bar{\mathbf{Y}}} \mapsto |w, x, D, \bar{y} \oplus f(D(x))\rangle_{\mathbf{WXD}\bar{\mathbf{Y}}}$$

which uses the same input/output registers $WX/\bar{Y}$ as O_0 and additionally employs D as a work register.

Let A be a quantum oracle algorithm (not necessarily unitary) making at most q_H queries to H/H_c and at most q_O queries to O_0/O_1 . For an arbitrary classical event Ev and a random z distributed according to $\mathfrak{D}_z$, define

$$P_{\text{left}} := \Pr[Ev : A^{|H\rangle, |O_0\rangle}(z)] \text{ and } P_{\text{right}} := \Pr[Ev : A^{|H_c\rangle, |O_1\rangle}(z)] ,$$

where the probability is taken over $z \leftarrow \mathfrak{D}_z$ and the internal randomness of A (including all quantum measurements). Setting $\Gamma_S := \max_{w,x} |S_{w,x}|$, we have

$$|P_{\text{left}} - P_{\text{right}}| \leq 6q_O \sqrt{\Gamma_S/|\mathcal{Y}|} \quad \text{and} \quad \left| \sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}} \right| \leq 6q_O \sqrt{\Gamma_S/|\mathcal{Y}|} .$$

The Lookup Lemma. It allows a simulator to locate database entries that satisfy complex conditions without relying on secret information (e.g., the secret key). It transforms a “compute-then-search” logic into a “direct-search under a new condition” logic. The transformation is undetectable whenever the fraction of codomain elements of H_c that satisfy exactly one of the two conditions is negligible.

Lemma 3.7 (Lookup Lemma). *Let $H_c : \mathcal{X} \rightarrow \mathcal{Y}$ be a compressed standard oracle with a database register D initialized to $\bigotimes_{i=1}^{q_H} |(\perp, 0^{\log|\mathcal{Y}|})\rangle$, and let O_0 be a quantum oracle acting on the input register W (defined over $\mathcal{W}$), the database register D , and an output register $\bar{Y}$ (defined over $\bar{\mathcal{Y}}$) as*

$$O_0 : |w, D, \bar{y}\rangle_{WD\bar{Y}} \mapsto |w, D, \bar{y} \oplus f_0(w, D)\rangle_{WD\bar{Y}}$$

with

$$f_0(w, D) = \begin{cases} g_{w,D}(x), & \text{if } (x, D(x)) \in S_w, \text{ where } x := d(w) , \\ g_{w,D}(\perp), & \text{otherwise} , \end{cases}$$

where each function $g_{w,D} : \mathcal{X} \cup \{\perp\} \rightarrow \bar{\mathcal{Y}}$ and each set $S_w \subseteq \mathcal{X} \times \mathcal{Y}$ are determined by (w, D) and by w , respectively, and $d : \mathcal{W} \rightarrow \mathcal{X}$ is a fixed function. Let O_1 be the quantum oracle

$$O_1 : |w, D, \bar{y}\rangle_{WD\bar{Y}} \mapsto |w, D, \bar{y} \oplus f_1(w, D)\rangle_{WD\bar{Y}}$$

with

$$f_1(w, D) = \begin{cases} g_{w,D}(x), & \text{if } \exists (x, y) \in D \text{ s.t. } (x, y) \in S'_w \\ & \text{and } \forall x' < x, (x', y') \in D \implies (x', y') \notin S'_w , \\ g_{w,D}(\perp), & \text{otherwise} , \end{cases}$$

which uses the same registers $WD\bar{Y}$ and the same function $g_{w,D}$ as O_0 . It should be note that the set $S'_w \subseteq \mathcal{X} \times \mathcal{Y}$ determined by w differs from S_w used in O_0 .

Let A be a quantum oracle algorithm (not necessarily unitary) making at most q_H queries to H_c and at most q_O queries to O_0/O_1 . For an arbitrary classical event Ev and a random z distributed according to $\mathfrak{D}_z$, define

$$P_{\text{left}} := \Pr[Ev : A^{|H_c\rangle, |O_0\rangle}(z)] \text{ and } P_{\text{right}} := \Pr[Ev : A^{|H_c\rangle, |O_1\rangle}(z)] ,$$

where the probability is taken over $z \leftarrow \mathfrak{D}_z$ and the internal randomness of A (including all quantum measurements). For each $w \in \mathcal{W}$, define the set

$$T_w := \{(x, y) \mid (x, y) \in S_w \text{ and } x = d(w)\} .$$

Then, let $R \subseteq \mathcal{X} \times \mathcal{Y}$ be the binary relation given by

$$R := \{(x, y) \mid \exists w \in \mathcal{W} \text{ s.t. } (x, y) \in T_w \Delta S'_w\} ,$$

where Δ denotes the symmetric difference, i.e., $T_w \Delta S'_w := (T_w \setminus S'_w) \cup (S'_w \setminus T_w)$. Let $\Gamma_R := \max_x |\{y \mid (x, y) \in R\}|$. Then, we have

$$|P_{\text{left}} - P_{\text{right}}| \leq 16(q_H + 1)\sqrt{\Gamma_R/|\mathcal{Y}|}, \quad \left| \sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}} \right| \leq 16(q_H + 1)\sqrt{\Gamma_R/|\mathcal{Y}|} .$$

The Puncturing Lemma. It provides a method for “selective forgetting” of specific database entries that were recorded by the compressed oracle query $H_c(x)$ at the beginning of a game. Such an operation is trivial in the ROM, where one can simply record only the adversary’s queries, but becomes challenging in the QROM, where the compressed oracle database indiscriminately records all queries in superposition. The modification is undetectable whenever the codomain of H_c is sufficiently large.

Lemma 3.8 (Puncturing Lemma). *Let $H_c : \mathcal{X} \rightarrow \mathcal{Y}$ be a compressed standard oracle with a database register D initialized to $\bigotimes_{i=1}^{q_H} |(\perp, 0^{\log|\mathcal{Y}|})\rangle$. For a fixed pair $(x^*, y^*) \in \mathcal{X} \times \mathcal{Y}$, define a punctured oracle*

$$H'_c : |x, y, D\rangle \mapsto \begin{cases} |x, y \oplus y^*, D\rangle, & \text{if } x = x^* , \\ H_c |x, y, D\rangle, & \text{otherwise} , \end{cases}$$

which answers queries to x^* without updating the database D . Let O_{x^*, y^*} be a quantum oracle acting on the input register W (defined over $\mathcal{W}$), the database D , and an output register $\bar{Y}$ (defined over $\bar{\mathcal{Y}}$) as

$$O_{x^*, y^*} : |w, D, \bar{y}\rangle_{W D \bar{Y}} \mapsto |w, D, \bar{y} \oplus f_{x^*, y^*}(w, D)\rangle_{W D \bar{Y}} ,$$

where $f_{x^*, y^*} : \mathcal{W} \times \mathcal{D} \rightarrow \bar{\mathcal{Y}}$ satisfies

$$f_{x^*, y^*}(w, D) = f_{x^*, y^*}(w, D \cup (x^*, y^*)) \text{ for every } D \text{ with } D(x^*) = \perp .$$

Let A be a quantum oracle algorithm (not necessarily unitary) making at most q_H queries to H_c/H'_c and at most q_O queries to O_{x^*, y^*} . For an arbitrary classical event Ev , define

$$\begin{aligned} P_{\text{left}} &:= \Pr[\text{Ev} : x^* \leftarrow \$ \mathcal{X}, y^* := H_c(x^*), z \leftarrow \mathfrak{D}_z, A^{|H_c\rangle, |O_{x^*, y^*}\rangle}(y^*, z)] , \\ P_{\text{right}} &:= \Pr[\text{Ev} : x^* \leftarrow \$ \mathcal{X}, y^* \leftarrow \$ \mathcal{Y}, z \leftarrow \mathfrak{D}_z, A^{|H'_c\rangle, |O_{x^*, y^*}\rangle}(y^*, z)] , \end{aligned}$$

where the probability is taken over all random coins. Then, we have

$$|P_{\text{left}} - P_{\text{right}}| \leq 2q_O/\sqrt{|\mathcal{Y}|} \quad \text{and} \quad \left| \sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}} \right| \leq 2q_O/\sqrt{|\mathcal{Y}|} .$$

3.2.4 QROM Security of NTRE.CCAKEM

In this section we analyze the QROM security of NTRE.CCAKEM, which is established by Theorem 3.3.

Games G_0 to G_4 :	12 : if $x = r^*$ then
1 : $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda), r^* \leftarrow \$ \mathcal{M}_2$	return $ x, y \oplus (m^*, \rho^*, k_0)\rangle \parallel_{G_2-G_3}$
2 : $F \leftarrow \$ \Omega_F, H \leftarrow \$ \Omega_H$	13 : perform $H_c \parallel_{G_1-G_4}$
3 : $(m_1^*, \rho^*, k_0) := H(r^*) \parallel_{G_0-G_1}$	Decapsulation Oracle $O_{\text{Dec}}(c)$:
4 : $(m_1^*, \rho^*, k_0) \leftarrow \$ \mathcal{M}_1 \times \mathcal{R} \times \mathcal{K} \parallel_{G_2-G_4}$	14 : if $c = c^*$ then : return $\perp$
5 : $m_2^* := r^* \oplus F(m_1^*)$	15 : $m_1 \parallel m_2 := \text{Dec}'(\text{sk}, c) \parallel_{G_2}$
6 : $c^* := \text{Enc}'(\text{pk}, m_1^* \parallel m_2^*; \rho^*)$	16 : if $m_1 \parallel m_2 = \perp$ then
7 : $b \leftarrow \$ \{0, 1\}, k_1 \leftarrow \$ \mathcal{K}$	return $\perp \parallel_{G_0-G_2}$
8 : $\hat{b} \leftarrow \mathcal{A}^{[F], [H], O_{\text{Dec}}}(\text{pk}, c^*, k_b)$	17 : $r := m_2 \oplus F(m_1) \parallel_{G_0-G_2}$
9 : return $[b = \hat{b}]$	18 : $(m'_1, \rho, k) := H(r) \parallel_{G_0}$
Random Oracle $F(x, y\rangle)$:	19 : $(m'_1, \rho, k) := D_H(r) \parallel_{G_1-G_2}$
10 : return $ x, y \oplus F(x)\rangle$	20 : if $m_1 = m'_1$ and
Random Oracle $H(x, y\rangle)$:	$\text{Enc}'(\text{pk}, m'_1 \parallel m_2; \rho) = c$ then
11 : return $ x, y \oplus H(x)\rangle \parallel_{G_0}$	return $k \parallel_{G_0-G_2}$
	21 : return $\perp \parallel_{G_0-G_2}$
	22 : return $f_1(c, D_H) \parallel_{G_3-G_4}$

Figure 6: Games G_0 to G_4 for the proof of Theorem 3.3.

Theorem 3.3 (QROM Security of KEM). *Assume that $F : \mathcal{M}_1 \rightarrow \mathcal{M}_2$ and $H : \mathcal{M}_2 \rightarrow \mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}$ are modeled as quantum-accessible random oracles. For any efficient adversary $\mathcal{A}$ with quantum access to the random oracles F and H that attacks the IND-CCA2 security of $\text{KEM} := \text{FOAC}'[\text{PKE}', F, H]$, there exist efficient adversaries $\mathcal{B}$ against the OW-CPA security of PKE' , such that*

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}(\mathcal{A}) \leq 4(q_F + q_H + 1)^{1.5} \sqrt{\text{Adv}_{\text{PKE}'}^{\text{OW-CPA}}(\mathcal{B})} + 16(q_H + 1)\sqrt{\delta_{ac}} + \frac{8q_D}{\sqrt{|\mathcal{M}_1|}} + \Delta,$$

where q_F, q_H, q_D are the bounded numbers of $\mathcal{A}$'s queries to the random oracles F, H , and the decapsulation oracle, respectively, and $\Delta \leq 24(q_H + 1)/\sqrt{|\mathcal{M}_2|}$.

Proof Sketch. The proof of Theorem 3.3 follows the high-level strategy of its ROM counterpart and lifts it to the QROM by combining the lemmas introduced in Section 3.2.3 with suitable One-Way to Hiding lemmas.

In more detail, Games G_0 – G_3 aim to make the decapsulation oracle $\mathcal{O}_{\text{Dec}}$ simulatable without the secret key sk by means of the Switching Lemma and the Lookup Lemma; specifically, the Puncturing Lemma is applied in Game G_2 to remove the entries inserted by the challenger from the database register D_H . In Game G_4 , the random oracle H is reprogrammed using the Semi-Classical One-Way to Hiding Lemma (Lemma 3.2), rendering the challenge keys k_0 and k_1 independent and uniformly random of all other oracle responses and public values. Finally, Games G_5 – G_6 further reduce the security to the OW-CPA security of the underlying PKE.

Proof (Theorem 3.3). We define a sequence of games G_j , played by the challenger and the adversary $\mathcal{A}$, as shown in Fig. 6 for $j = 0, \dots, 4$. In each game, let b be the random bit sampled by the challenger and $\hat{b}$ the bit output by $\mathcal{A}$. Denote by W_j the event that $b = \hat{b}$ in game G_j .

Game G_0 . At the beginning of the game, the challenger randomly samples two functions $F \leftarrow \$ \Omega_F$ and $H \leftarrow \$ \Omega_H$ to model the random oracles F and H , respectively. We can see that the behavior of the challenger in Game G_0 is identical to that in the IND-CCA2 game for $\text{KEM} := \text{FOAC}'[\text{PKE}', F, H]$. Hence,

$$|\Pr[W_0] - 1/2| = \text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}(\mathcal{A}) .$$

Game G_1 . This game first simulates the random oracle H using the compressed oracle H_c with database register D_H (as shown in line 13 of Fig. 6), and then modifies the decapsulation oracle. Given a query $c \neq c^*$, after computing $m_1 \| m_2 := \text{Dec}'(\text{sk}, c)$ ($\neq \perp$) and $r := m_2 \oplus F(m_1)$, the behavior of the decapsulation oracle in Game G_0 is equivalent to first applying a unitary operator

$$|(c, m_1, m_2), r, 0^{\log|\mathcal{K}|}\rangle \mapsto \begin{cases} |(c, m_1, m_2), r, 0^{\log|\mathcal{K}|} \oplus H(r)[2]\rangle, & \text{if } H(r) \in S_{(c, m_1, m_2), r}, \\ |(c, m_1, m_2), r, 0^{\log|\mathcal{K}|} \oplus \perp\rangle, & \text{otherwise,} \end{cases}$$

where $S_{(c, m_1, m_2), r} := \{(m'_1, \rho, k) \mid m'_1 = m_1 \wedge \text{Enc}'(\text{pk}, m'_1 \| m_2; \rho) = c\}$, and then measuring the last register in the computational basis to obtain the output of O_{Dec} . In this game, we replace $H(r)$ in the unitary with $D_H(r)$, where $|D_H\rangle$ is the computational basis state of D_H . We indicate this change in line 19 of Fig. 6 by writing $(m'_1, \rho, k) := D_H(r)$, meaning that the tuple (m'_1, ρ, k) is now from the value stored at position r in D_H , rather than from $H(r)$. Since

$$\begin{aligned} \Gamma_{S_{(c, m_1, m_2), r}} &= \max_{(c, m_1, m_2), r} |\{(m'_1, \rho, k) \mid m'_1 = m_1 \wedge \text{Enc}'(\text{pk}, m'_1 \| m_2; \rho) = c\}| \\ &\leq \max_{(c, m_1, m_2), r} |\{(m'_1, \rho, k) \mid m'_1 = m_1\}| \\ &= |\{m_1\} \times \mathcal{R} \times \mathcal{K}| = |\mathcal{R} \times \mathcal{K}| , \end{aligned}$$

by the Switching Lemma (Lemma 3.6), we have

$$|\Pr[W_1] - \Pr[W_0]| \leq 6q_D \sqrt{\Gamma_{S_{(c, m_1, m_2), r}} / |\mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}|} = 6q_D \sqrt{1/|\mathcal{M}_1|} .$$

Game G_2 . Note that in Game G_1 , given a query $c \neq c^*$ and the database register D_H , after computing $m_1 \| m_2 := \text{Dec}'(\text{sk}, c)$ ($\neq \perp$) and $r := m_2 \oplus F(m_1)$, the decapsulation oracle can be viewed as applying a unitary operator

$$|(c, m_1, m_2, r), D_H, 0^{\log|\mathcal{K}|}\rangle \mapsto |(c, m_1, m_2, r), D_H, 0^{\log|\mathcal{K}|} \oplus f((c, m_1, m_2, r), D_H)\rangle ,$$

where

$$f((c, m_1, m_2, r), D_H) = \begin{cases} D_H(r)[2], & \text{if } m'_1 = m_1 \text{ and } \text{Enc}'(\text{pk}, m'_1 \| m_2; \rho) = c \\ & \text{with } (m'_1, \rho, k) := D_H(r) , \\ \perp, & \text{otherwise,} \end{cases}$$

and then measuring the last register in the computational basis to obtain the response of O_{Dec} . Observe that by the definition of f we have

$$f((c, m_1, m_2, r), D_H) = f((c, m_1, m_2, r), D_H \cup \{(r^*, (m_1^*, \rho^*, k_0))\})$$

for every D_H with $D_H(r^*) = \perp$. Otherwise, there would exist some $c \neq c^*$ such that $r^* = m_2 \oplus F(m_1^*)$ (implying $m_2 = m_2^*$) and $\text{Enc}'(\text{pk}, m_1^* \| m_2; \rho^*) = c$, which contradicts

$c \neq c^*$. Hence the function f satisfies the conditions required by the Puncturing Lemma (Lemma 3.8).

Consequently, in this game we sample (m_1^*, ρ^*, k_0) uniformly at random (as shown in line 4 of Fig. 6) and modify the behavior of the random oracle H to

$$|x, y, D_H\rangle \mapsto \begin{cases} |x, y \oplus (m_1^*, \rho^*, k_0), D_H\rangle, & \text{if } x = r^*, \\ H|x, y, D_H\rangle, & \text{otherwise,} \end{cases}$$

i.e., we add line 12 in Fig. 6. By the Puncturing Lemma (Lemma 3.8), we have

$$|\Pr[W_2] - \Pr[W_1]| \leq 2q_D / \sqrt{|\mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}|}.$$

Game G_3 . Note that in Game G_2 , after receiving a query $c \neq c^*$ and given the database register D_H , the decapsulation oracle can be viewed as first applying a unitary operator

$$|(c, D_H, 0^{\log|\mathcal{K}|})\rangle \mapsto |c, D_H, 0^{\log|\mathcal{K}|} \oplus f_0(c, D_H)\rangle,$$

with³

$$f_0(c, D_H) = \begin{cases} D_H(r)[2], & \text{if } (r, D_H(r)) \in S_c, \\ \text{where } r = \text{Dec}'(\text{sk}, c)[1] \oplus F(\text{Dec}'(\text{sk}, c)[0]), \\ \perp, & \text{otherwise,} \end{cases}$$

where $S_c := \{(r, (m'_1, \rho, k)) \mid m'_1 = \text{Dec}'(\text{sk}, c)[0] \wedge \text{Enc}'(\text{pk}, m'_1 \parallel (r \oplus F(m'_1)); \rho) = c\}$. Here, $\text{Dec}'(\text{sk}, c)[0]$ (resp. $\text{Dec}'(\text{sk}, c)[1]$) denotes the first part m_1 (resp. the second part m_2) for $\text{Dec}'(\text{sk}, c) = m_1 \parallel m_2$. The oracle then measures the last register in the computational basis to obtain the classical response of O_{Dec} . In this game, we replace f_0 by

$$f_1(c, D_H) = \begin{cases} D_H(r)[2], & \text{if } \exists (r, D_H(r)) \in S'_c, \\ \text{and } \forall r' < r, (r', D_H(r')) \notin S'_c, \\ \perp, & \text{otherwise,} \end{cases}$$

where $S'_c := \{(r, (m'_1, \rho, k)) \mid \text{Enc}'(\text{pk}, m'_1 \parallel (r \oplus F(m'_1)); \rho) = c\}$.

To apply the Lookup Lemma (Lemma 3.7), we first define, for each c , the set

$$T_c := \{(r, (m'_1, \rho, k)) \in S_c \mid r = \text{Dec}'(\text{sk}, c)[1] \oplus F(\text{Dec}'(\text{sk}, c)[0])\}.$$

We then introduce the binary relation

$$R := \{(r, (m'_1, \rho, k)) \mid \exists c \text{ s.t. } (r, (m'_1, \rho, k)) \in T_c \Delta S'_c\}.$$

For every $(r, (m'_1, \rho, k)) \in T_c$, the relation $\text{Enc}'(\text{pk}, m'_1 \parallel (r \oplus F(m'_1)); \rho) = c$ necessarily holds, which implies $(r, (m'_1, \rho, k)) \in S'_c$; consequently, $T_c \setminus S'_c = \emptyset$. On the other hand, for any $(r, (m'_1, \rho, k)) \in S'_c$ with $(r, (m'_1, \rho, k)) \notin T_c$, we must have

$$\text{Dec}'(\text{sk}, \text{Enc}'(\text{pk}, m'_1 \parallel (r \oplus F(m'_1)); \rho)) \neq m'_1 \parallel (r \oplus F(m'_1)).$$

³For technical convenience, we extend the definition of the random oracle F by the convention $F(\perp) := \perp$.

Therefore,

$$\begin{aligned}
\Gamma_R &= \max_r |\{(m'_1, \rho, k) \mid (r, (m'_1, \rho, k)) \in R\}| \\
&\leq \max_r |\{(m'_1, \rho, k) \mid \exists c \text{ s.t. } (r, (m'_1, \rho, k)) \in T_c \setminus S'_c\}| \\
&\quad + \max_r |\{(m'_1, \rho, k) \mid \exists c \text{ s.t. } (r, (m'_1, \rho, k)) \in S'_c \setminus T_c\}| \\
&\leq \max_r |\Theta_r| \cdot |\mathcal{K}|,
\end{aligned}$$

where the set $\Theta_r := \{(m'_1, \rho) \mid \text{Dec}'(\text{sk}, \text{Enc}'(\text{pk}, m'_1 \parallel (r \oplus F(m'_1)); \rho)) \neq m'_1 \parallel (r \oplus F(m'_1))\}$. For a fixed key pair (pk, sk) and a fixed function F that is used to simulated the random oracle F , the Lookup Lemma (Lemma 3.7) yields

$$\begin{aligned}
&|\Pr[W_3 : (\text{pk}, \text{sk}, F)] - \Pr[W_2 : (\text{pk}, \text{sk}, F)]| \\
&\leq 16(q_H + 1) \sqrt{\Gamma_R / |\mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}|} \leq 16(q_H + 1) \sqrt{\max_r |\Theta_r| / |\mathcal{M}_1 \times \mathcal{R}|}.
\end{aligned}$$

For a given (pk, sk) , define

$$\delta(\text{pk}, \text{sk}) := \Pr[\text{Dec}'(\text{sk}, \text{Enc}'(\text{pk}, m_1 \parallel m_2; \rho)) \neq m_1 \parallel m_2],$$

where the probability is taken over $m_1 \parallel m_2 \leftarrow \$ \mathcal{M}_1 \times \mathcal{M}_2$ and $\rho \leftarrow \$ \mathcal{R}$. Note that for any fixed $r \in \mathcal{M}_2$ and any $m_1 \in \mathcal{M}_1$, the variable $r \oplus F(m_1)$ (with $F \leftarrow \$ \Omega_F$) is also uniformly distributed over $\mathcal{M}_2$. Hence, we have

$$\mathbb{E}_{F \leftarrow \$ \Omega_F} [|\Theta_r| / |\mathcal{M}_1 \times \mathcal{R}|] \leq \delta(\text{pk}, \text{sk}) \quad \text{for every } r.$$

Thus, by averaging over $F \leftarrow \$ \Omega_F$ and using Jensen's inequality, we can obtain

$$\begin{aligned}
&|\Pr[W_3 : (\text{pk}, \text{sk})] - \Pr[W_2 : (\text{pk}, \text{sk})]| \\
&\leq 16(q_H + 1) \mathbb{E}_{F \leftarrow \$ \Omega_F} [\sqrt{\max_r |\Theta_r| / |\mathcal{M}_1 \times \mathcal{R}|}] \\
&\leq 16(q_H + 1) \sqrt{\mathbb{E}_{F \leftarrow \$ \Omega_F} [\max_r |\Theta_r| / |\mathcal{M}_1 \times \mathcal{R}|]} \leq 16(q_H + 1) \sqrt{\delta(\text{pk}, \text{sk})}.
\end{aligned}$$

Finally, by averaging over the key pair $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$ and applying Jensen's inequality once more, we have

$$\begin{aligned}
&|\Pr[W_3] - \Pr[W_2]| \leq 16(q_H + 1) \mathbb{E}_{(\text{pk}, \text{sk})} [\sqrt{\delta(\text{pk}, \text{sk})}] \\
&\leq 16(q_H + 1) \sqrt{\mathbb{E}_{(\text{pk}, \text{sk})} [\delta(\text{pk}, \text{sk})]} \stackrel{(a)}{=} 16(q_H + 1) \sqrt{\delta_{ac}},
\end{aligned}$$

where (a) follows from the definition of the δ_{ac} -average-case correctness of PKE' .

Note that from this game onward, O_{Dec} can be simulated without sk .

Game G_4 . In this game, we remove line 12 of Fig. 6, which is equivalent to reprogramming the random oracle H at the point r^* . Now, we can construct two oracle algorithms, $A^{(F), (H), O_{\text{Dec}}}(z)$ and $A^{(F), (H'), O_{\text{Dec}}}(z)$, to simulate Games G_3 and G_2 , respectively, and then apply the Semi-Classical One-Way to Hiding Lemma (Lemma 3.2) to bound $|\Pr[W_4] - \Pr[W_3]|$.

Let $z := (\text{pk}, c^*, k_0, k_1, b)$, where $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$, $r^* \leftarrow \$ \mathcal{M}_2$, $(m_1^*, \rho^*, k_0) \leftarrow \$ \mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}$, $m_2^* := r^* \oplus F(m_1^*)$, $c^* := \text{Enc}'(\text{pk}, m_1^* \parallel m_2^*; \rho^*)$, $k_1 \leftarrow \$ \mathcal{K}$, and $b \leftarrow \$ \{0, 1\}$. Let the random oracle H be simulated by the compressed oracle H_c (as in Game G_4), and let H' be an oracle such that $H'(r) = H(r)$ for all $r \neq r^*$ and $H'(r^*) = (m_1^*, \rho^*, k_0)$ (as in Game G_3). The behaviors of the random oracle F and the decapsulation oracle O_{Dec} are the same as in Game G_3 . The algorithm $A^{(F), (H), O_{\text{Dec}}}(z)$ (or $A^{(F), (H'), O_{\text{Dec}}}(z)$) runs $\hat{b} \leftarrow \mathcal{A}^{(F), (H), O_{\text{Dec}}}(z)$

Games G_5 to G_6 :	Random Oracle $F(x, y\rangle)$:
1 : $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$	10 : return $ x, y \oplus F(x)\rangle$
2 : $F \leftarrow \$ \Omega_F$	
3 : $(m_1^*, \rho^*, k_0) \leftarrow \$ \mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}$	Punctured Oracle $H \setminus \{r^*\}(x, y\rangle)$:
4 : $r^* \leftarrow \$ \mathcal{M}_2, m_2^* := r^* \oplus F(m_1^*) \quad // G_5$	11 : perform $\mathcal{O}_{\{r^*\}}^{SC} \quad // G_5$
5 : $m_2^* \leftarrow \$ \mathcal{M}_2 \quad // G_6$	12 : perform $\mathcal{O}_{\{m_2^* \oplus F(m_1^*)\}}^{SC} \quad // G_6$
6 : $c^* := \text{Enc}'(\text{pk}, m_1^* \ m_2^*; \rho^*) \quad // G_5 - G_6$	13 : perform H_c
7 : $b \leftarrow \$ \{0, 1\}, k_1 \leftarrow \$ \mathcal{K}$	
8 : $\hat{b} \leftarrow \mathcal{A}^{[F], H \setminus \{r^*\}\rangle, O_{\text{Dec}}}(\text{pk}, c^*, k_b)$	Decapsulation Oracle $O_{\text{Dec}}(c)$:
9 : if Find then : return 1	14 : if $c = c^*$ then : return $\perp$
else : return 0	15 : return $f_1(c, D_H)$

Figure 7: Games G_5 to G_6 for the proof of Theorem 3.3.

(or $\hat{b} \leftarrow \mathcal{A}^{[F], |H'\rangle, O_{\text{Dec}}}(z)$) and then outputs $[b = \hat{b}]$. Note that the behaviors of H and O_{Dec} in Games G_4 and G_3 are identical. It is easy to see that

$$\Pr[1 \leftarrow A^{[F], |H\rangle, O_{\text{Dec}}}(z)] = \Pr[W_4] \text{ and } \Pr[1 \leftarrow A^{[F], |H'\rangle, O_{\text{Dec}}}(z)] = \Pr[W_3] .$$

By the Semi-Classical One-Way to Hiding lemma (Lemma 3.2), we can obtain⁴

$$\begin{aligned} & \left| \Pr[1 \leftarrow A^{[F], |H\rangle, O_{\text{Dec}}}(z)] - \Pr[1 \leftarrow A^{[F], |H'\rangle, O_{\text{Dec}}}(z)] \right| \\ & \leq 2\sqrt{(q_H + 1) \Pr[\text{Find}_{r^*} : A^{[F], |H \setminus \{r^*\}\rangle, O_{\text{Dec}}}(z)]} . \end{aligned}$$

where $H \setminus \{r^*\}$ denotes a punctured oracle that, before each query to H , first applies a semi-classical oracle $\mathcal{O}_{\{r^*\}}^{SC}$ (as defined in Definition 3.11) to the input, and then proceeds with the usual query to H ; Find_{r^*} is the classical event that the measurement of $\mathcal{O}_{\{r^*\}}^{SC}$ ever outputs 1 (as defined in Definition 3.11).

Note that in this game, the uniformly random k_0 and k_1 are independent of all other oracle answers and public values; therefore, the bit b is uniformly random and independent of $\mathcal{A}$'s view. Consequently,

$$\Pr[W_4] = 1/2 .$$

We next define a sequence of games G_j (for $j = 5, 6$) played between the challenger and the adversary $\mathcal{A}$, as depicted in Fig. 7. In each game, Find denotes the event that the measurement of the semi-classical oracle inside the punctured oracle $H \setminus \{r^*\}$ ever outputs 1. Let Z_j be the event that Find occurs in Game G_j .

Game G_5 . This game is defined as in Fig. 7. Compared to the challenger in Game G_4 , it additionally samples $\hat{m}_1^* \leftarrow \$ \mathcal{M}_1$, which is never used in the game, and swaps the order of sampling (m_1^*, ρ^*, k_0) and r^* . It is easy to see that these changes do not affect the behavior of the challenger from $\mathcal{A}$'s view. By the definitions of the punctured oracle $H \setminus \{r^*\}$ and the event Find in Game G_5 (depicted in Fig. 7), we can see that

$$\Pr[Z_5] = \Pr[\text{Find}_{r^*} : A^{[F], |H \setminus \{r^*\}\rangle, O_{\text{Dec}}}(z)] .$$

⁴When applying Lemma 3.2, F and O_{Dec} can be treated as $\mathcal{A}$'s internal oracles.

Games G_6^A to G_7^A :	Random Oracle $F(x, y\rangle)$:
1 : $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$	10 : return $ x, y \oplus F(x)\rangle \quad // \quad G_6^A$
2 : $F \leftarrow \$ \Omega_F$	11 : perform $F_c \quad // \quad G_7^A$
3 : $(m_1^*, \rho^*, k_0) \leftarrow \$ \mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}$	
4 : $m_2^* \leftarrow \$ \mathcal{M}_2$	Random Oracle $H(x, y\rangle)$:
5 : $c^* := \text{Enc}'(\text{pk}, m_1^* \ m_2^*; \rho^*)$	12 : perform H_c
6 : $b \leftarrow \$ \{0, 1\}, k_1 \leftarrow \$ \mathcal{K}$	
7 : $\hat{r} \leftarrow B^{[F], [H], O_{\text{Dec}}}(\text{pk}, c^*, k_b)$	Decapsulation Oracle $O_{\text{Dec}}(c)$:
8 : return $[\hat{r} = (m_2^* \oplus F(m_1^*))] \quad // \quad G_6^A$	13 : if $c = c^*$ then : return $\perp$
9 : return $[\hat{r} = (m_2^* \oplus D_F(m_1^*))] \quad // \quad G_7^A$	14 : return $f_1(c, D_H)$

Figure 8: Games G_6^A to G_7^A for the proof of Theorem 3.3.

Game G_6 . In this game, we make a *conceptual* change to the challenger's description. Recall that in Game G_5 the challenger samples $r^* \leftarrow \$ \mathcal{M}_2$ uniformly and then sets $m_2^* := r^* \oplus F(m_1^*)$ (line 4 of Fig. 7). Because r^* is uniform and independent of $F(m_1^*)$, the value m_2^* is itself uniformly distributed over $\mathcal{M}_2$. Hence, we can replace that two-step procedure with the direct sampling $m_2^* \leftarrow \$ \mathcal{M}_2$ (now line 5 of Fig. 7), which also implicitly defines $r^* := m_2^* \oplus F(m_1^*)$. Consequently, the semi-classical oracle $\mathcal{O}_{\{r^*\}}^{SC}$ (line 11) is modified to $\mathcal{O}_{\{m_2^* \oplus F(m_1^*)\}}^{SC}$ (line 12). Since this alteration is purely a reformulation of the same process, we have

$$\Pr[Z_6] = \Pr[Z_5] .$$

Now we can bound $\Pr[Z_6]$ if PKE' is OW-CPA. Let $B^{[F], [H], O_{\text{Dec}}}(z')$ be an oracle algorithm that takes $z' := (\text{pk}, c^*, k_b)$ as input, randomly picks $i^* \leftarrow \$ \{1, \dots, q_H\}$, runs $\mathcal{A}^{[F], [H], O_{\text{Dec}}}(z)$ until (just before) the i^* -th query to the random oracle H , measures the input register in the computational basis, and outputs the measurement result. Here H behaves the same as $H \setminus \{r^*\}$, except that it does not query the semi-classical oracle $\mathcal{O}_{\{m_2^* \oplus F(m_1^*)\}}^{SC}$. By Lemma 3.3, we have

$$\Pr[Z_6] \leq 4q_H \cdot \Pr[\hat{r} = (m_2^* \oplus F(m_1^*)) : \hat{r} \leftarrow B^{[F], [H], O_{\text{Dec}}}(z')] .$$

Consider a game G_6^A played between the challenger and B , as shown in Fig. 8, and let Z_j^A be the event that the challenger outputs 1 at the end of game G_j^A . It is clear that

$$\Pr[Z_6^A] = \Pr[\hat{r} = (m_2^* \oplus F(m_1^*)) : \hat{r} \leftarrow B^{[F], [H], O_{\text{Dec}}}(z')] .$$

Now we modify G_6^A to obtain G_7^A , as shown in Fig. 8. In this game, we further simulate the random oracle F using the compressed oracle F_c with database register D_F (line 11 of Fig. 8). Note that in Game G_6^A , after the oracle algorithm B outputs $\hat{r}$, the challenger is equivalent to applying the unitary operator

$$|(m_2^*, \hat{r}), m_1^*, 0\rangle \mapsto \begin{cases} |(m_2^*, \hat{r}), m_1^*, 1\rangle, & \text{if } F(m_1^*) \in S_{(m_2^*, \hat{r}), m_1^*} , \\ |(m_2^*, \hat{r}), m_1^*, 0\rangle, & \text{otherwise} , \end{cases}$$

where $S_{(m_2^*, \hat{r}), m_1^*} := \{u \mid m_2^* \oplus u = \hat{r}\}$, and then measures the last register in the computational basis to obtain the final outcome of the game. We replace $F(m_1^*)$ in this

unitary by $D_F(m_1^*)$, where $|D_F\rangle$ is the computational-basis state of D_F . We denote the challenger's check on $\hat{r}$ output by B in Game G_7^A as $\hat{r} = (m_2^* \oplus D_F(m_1^*))$ (line 9 of Fig. 8). Clearly, exactly one value $u = m_2^* \oplus \hat{r}$ satisfies $m_2^* \oplus u = \hat{r}$; hence $\Gamma_{S(m_2^*, \hat{r}), m_1^*} = \max_{(m_2^*, \hat{r}), m_1^*} |S(m_2^*, \hat{r}), m_1^*| = 1$. Therefore, by the Switching Lemma (Lemma 3.6) we obtain

$$\left| \sqrt{\Pr[Z_7^A]} - \sqrt{\Pr[Z_6^A]} \right| \leq 6\sqrt{1/|\mathcal{M}_2|}.$$

By the basic principles of quantum computation, the challenger's verification of the value $\hat{r}$ output by B in Game G_7^A is equivalent to first measuring the register D_F in the computational basis, obtaining a classical database D_F , and then checking whether $m_2^* \oplus D_F(m_1^*) = \hat{r}$. If the equality holds, it means there exists an entry $(m_1^*, D_F(m_1^*))$ in D_F such that $m_2^* = \hat{r} \oplus D_F(m_1^*)$.

Based on this observation, and since now the values m_1^*, m_2^*, ρ^* are used only to compute $c^* := \text{Enc}'(\text{pk}, m_1^* \| m_2^*; \rho^*)$ while the challenger's simulation does not require sk , we can construct an adversary $\mathcal{B}$ that attacks the OW-CPA security of PKE' whenever the event Z_7^A occurs. Concretely, upon receiving the challenge (pk, c^*) from its OW-CPA challenger, $\mathcal{B}$ samples $k_0, k_1 \leftarrow \$ \mathcal{K}$ and $b \leftarrow \$ \{0, 1\}$, forms $z' := (\text{pk}, c^*, k_b)$, and interacts with $B^{(F), (H), O_{\text{Dec}}}(z')$ by simulating the oracles F, H, O_{Dec} exactly as in Game G_7^A . After B outputs $\hat{r}$, $\mathcal{B}$ measures D_F in the computational basis to obtain a classical D_F , randomly chooses a pair (m_1, u) from D_F , computes $m_2 := \hat{r} \oplus u$, and outputs $m_1 \| m_2$ as the solution to the OW-CPA challenge. Clearly, if the event $\hat{r} = (m_2^* \oplus D_F(m_1^*))$ holds and $\mathcal{B}$ happens to pick the correct pair $(m_1^*, D_F(m_1^*))$, then $m_1 \| m_2$ is indeed a valid solution. This occurs with probability $\Pr[Z_7^A]/|D_F|$.

Recall that during the simulation of the decapsulation oracle, each evaluation of f_1 requires queries to the random oracle F for every entry in D_H with $D_H(r) \neq \perp$. Since B makes at most q_H queries to H , we have $|D_H| \leq q_H$. Moreover, by the property of the compressed oracle, all first components m_1 stored in D_F are distinct. Consequently, the number of entries added to D_F via the F -queries inside O_{Dec} is at most q_H ; together with at most q_F direct queries of B to F , we obtain $|D_F| \leq q_F + q_H$. Therefore,

$$\Pr[Z_7^A]/(q_F + q_H) \leq \text{Adv}_{\text{PKE}'}^{\text{OW-CPA}}(\mathcal{B}).$$

Putting all bounds together, we have

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}(\mathcal{A}) &\leq 2\sqrt{4q_H(q_H+1)} \left(\sqrt{(q_F + q_H) \text{Adv}_{\text{PKE}'}^{\text{OW-CPA}}(\mathcal{B})} + \frac{6}{\sqrt{|\mathcal{M}_2|}} \right) \\ &\quad + 16(q_H+1)\sqrt{\delta_{ac}} + \frac{6q_D}{\sqrt{|\mathcal{M}_1|}} + \frac{2q_D}{\sqrt{|\mathcal{M}_1 \times \mathcal{R} \times \mathcal{K}|}} \\ &\leq 4(q_F + q_H + 1)^{1.5} \sqrt{\text{Adv}_{\text{PKE}'}^{\text{OW-CPA}}(\mathcal{B})} + 16(q_H + 1)\sqrt{\delta_{ac}} \\ &\quad + 8q_D/\sqrt{|\mathcal{M}_1|} + 24(q_H + 1)/\sqrt{|\mathcal{M}_2|}. \end{aligned}$$

□

3.3 Concrete Security

To evaluate the concrete security of NTRE, we estimate the hardness of the $\mathcal{R}$ -LWE $_\eta$ and $\mathcal{R}$ -NTRU $_\eta$ problems using the Lattice Estimator [1]⁵ and the NTRU Estimator from [6],

⁵<https://github.com/malb/lattice-estimator>

respectively. The Lattice Estimator accounts for the most effective lattice reduction attacks, including the BKZ algorithm [9] and its optimized primal [2] and dual [14] variants. The NTRU Estimator additionally incorporates specialized attacks against NTRU lattices, notably Howgrave-Graham’s hybrid attack [13].

Our parameter sets, listed in Table 1, are derived directly from the estimates provided by these tools. For each targeted security level, we select (n, q) such that both estimators report security strength meeting the NGCC requirements against all the aforementioned attacks. Thus, the chosen parameters offer concrete security at the claimed levels, fully resisting the known attacks covered by the two estimators.

4 Performance Evaluation

All benchmarks were obtained on a single core (bound via `taskset -c 0`) of an **Intel(R) Core(TM) Ultra 9 275HX** processor with a base clock frequency of **2.70 GHz** (Turbo Boost enabled), running **WSL 2 Ubuntu 26.04** and equipped with **64 GB of RAM**. The processor supports a wide range of instruction sets, including MMX, SSE, SSSE3, EM64T, AES, AVX, AVX2, AVX-VNNI, FMA3, and SHA. Our optimized implementation specifically leverages **AVX2, AVX-VNNI, FMA3, and SHA** to accelerate the number-theoretic transform (NTT), point-wise multiplications, and hashing operations, which are the core bottlenecks in polynomial arithmetic and KEM routines. Implementations were compiled using **GCC version 15.2.0** with **CMake 4.2.3** as the build tool, and cycle counts were measured using the **x86 rdtsc counter**. Table 4 lists the clock cycles of our optimized C implementation of NTRE.

Table 4: Computational and Communication Costs of NTRE.CCAKEM

NTRE-2593-648					
Size (bytes)		C		AVX2	
<i>sk</i>	1976	KGen	90,555	KGen	56,609
<i>pk</i>	972	Encaps	72,201	Encaps	58,438
<i>ct</i>	972	Decaps	69,989	Decaps	49,684
NTRE-2917-1296					
Size (bytes)		C		AVX2	
<i>sk</i>	3920	KGen	154,923	KGen	81,890
<i>pk</i>	1944	Encaps	129,226	Encaps	80,914
<i>ct</i>	1944	Decaps	133,626	Decaps	62,771
NTRE-3457-2304					
Size (bytes)		C		AVX2	
<i>sk</i>	6944	KGen	376,672	KGen	137,912
<i>pk</i>	3456	Encaps	351,723	Encaps	171,718
<i>ct</i>	3456	Decaps	360,075	Decaps	140,883

4.1 Performance Analysis

The efficiency of the NTRE key encapsulation mechanism stems from a combination of compact parameter selection, an average-case-friendly KEM transform, and a highly optimized polynomial arithmetic backend.

Compact PKE via $p = 2$ and efficient encoding. The underlying NTRE.CPAPKE is instantiated with the smallest possible prime $p = 2$, which directly minimizes the noise growth in the decryption equation $\mathbf{f} \cdot \mathbf{c} = 2(\mathbf{g}\mathbf{r} + \mathbf{f}'\mathbf{m}) + \mathbf{m}$. This allows us to choose significantly smaller n and q while keeping the decryption failure probability negligible. Message encoding is handled by CBD'_1 (Algorithm 4), which, together with a random seed r , maps a message $m \in \{0, 1\}^n$ to a polynomial $\mathbf{m} \in \mathcal{R}_q$ with coefficients in $\{-1, 0, 1\}$ following ψ_1 . This encoding not only matches the requirement of the RLWE-based encryption ($\mathbf{c} = \mathbf{r} \cdot \mathbf{h} + \mathbf{m}$) but also enables trivial recovery via $\mathbf{m} \bmod 2$ during decryption.

As a result, key generation requires only one polynomial inversion (for $\mathbf{f}^{-1}$) and one

multiplication (for $\mathbf{h} = 2\mathbf{g}/\mathbf{f}$), while encryption and decryption reduce to a single polynomial multiplication-addition and a single modular multiplication, respectively. This algebraic simplicity directly translates into compact communication sizes: as shown in Table 4, the public key and ciphertext are identically sized at **972 bytes** for NTRE-2593-648, **1944 bytes** for NTRE-2917-1296, and **3456 bytes** for NTRE-3457-2304. The secret key sizes scale to **1976**, **3920**, and **6944 bytes**, respectively. As can be seen from Table 5, NTRE achieves the smallest key and ciphertext sizes among all compared schemes at comparable security levels that satisfy the security requirements of the NGCC.

Table 5: Comparison of public/private key and ciphertext sizes between NTRE and other lattice-based post-quantum KEMs.

Scheme	Security level (bits)		$ pk $ (B)	$ ct $ (B)	$ sk $ (B)
	Classical	Quantum			
NTRE-2593-648	132	118	972	972	1976
NTRE-2917-1296	290	259	1944	1944	3920
NTRE-3457-2304	584	513	3456	3456	6944
NTRU ⁺ 768	156	139	1152	1152	2336
NTRU ⁺ 864	179	160	1296	1296	2624
NTRU ⁺ 1152	248	222	1728	1728	3488
ML-KEM-512	115	103	800	768	1632
ML-KEM-768	174	155	1184	1088	2400
ML-KEM-1024	241	215	1568	1568	3168

KEM transform with re-encryption check. To achieve IND-CCA2 security, we apply the FOAC' transform (Figure 1). Unlike some lightweight FO variants that omit re-encryption to save cycles, FOAC' *retains* the re-encryption check during decapsulation (see the right part of Figure 1, line 5) to explicitly detect invalid ciphertexts and thwart chosen-ciphertext attacks. Crucially, FOAC' removes the stringent γ -spread condition required by the original FOAC and relies only on the *average-case* decryption error. Since our $p = 2$ parameterization ensures that failures occur with negligible probability for random messages, the re-encryption step adds only minimal overhead, essentially one additional NTT-based polynomial multiplication, while providing strong CCA robustness. Thus, decapsulation consists of one decryption, two hash evaluations, one XOR/comparison, and one re-encryption, making it highly efficient in practice.

It is worth noting that although NTRU+ saves one re-encryption compared with NTRE, its decapsulation must compute $(\mathbf{c} - \mathbf{m}) \cdot \mathbf{h}^{-1}$ to recover $\hat{\mathbf{r}}$. The cost of this operation is essentially comparable to the re-encryption $\mathbf{h} \cdot \mathbf{r} + \mathbf{e}$ in NTRE. Hence, the re-encryption step imposes no additional computational overhead on the decapsulation speed of NTRE compared with NTRU+. Moreover, because NTRE does not require the public key $\mathbf{h}$ to be invertible, its key generation algorithm KGen is faster.

NTT-friendly ring and optimizations. All polynomial operations are accelerated using the number-theoretic transform (NTT) over the ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n - X^{n/2} + 1)$, where $n = 2^i 3^j$ and $q \equiv 1 \pmod{n}$. To eliminate redundant conversions, both the public

key $\mathbf{pk}$ and the secret key $\mathbf{sk}$ are stored directly in their NTT domain representation. The optimized AVX2 implementation further leverages the FMA3 and AVX-VNNI instruction sets to accelerate the underlying NTT and point-wise multiplications.

The performance gains are substantial. As reported in Table 4, for NTRE-2593-648, key generation drops from **90,555** to **56,609** cycles; for NTRE-2917-1296, encapsulation drops from **129,226** to **80,914** cycles; and for NTRE-3457-2304, decapsulation drops from **360,075** to **140,883** cycles. Overall, the combination of compact $p = 2$ parameters, the average-case-driven FOAC' transform, and the NTT/AVX2 backend allows NTRE to achieve scalable performance that grows roughly linearly with n , while maintaining moderate communication overhead.

It should be noted, as discussed in Section 1.4, that in compliance with the NGCC requirements we instantiate symmetric primitives such as the extendable-output function and hash functions using the API provided by ICCS. Hence, the reported cycle counts primarily reflect the speedup of our optimized implementation over the reference implementation, rather than the absolute performance of the scheme. The ICCS API is slower than mainstream primitives such as SHA-3; adopting more efficient symmetric primitives would therefore further improve the efficiency of our scheme.

4.2 Performance Characteristics

According to Table 4, the main performance characteristics of the NTRE key encapsulation mechanism can be summarized as follows.

In terms of communication overhead, both the public key and ciphertext sizes scale linearly with the parameter set: for NTRE-2593-648 they are each **972 bytes**; for NTRE-2917-1296, **1944 bytes**; and for NTRE-3457-2304, **3456 bytes**. The secret key sizes are consistently larger: **1976 bytes**, **3920 bytes**, and **6944 bytes** for the three parameter sets, respectively. Hence, the communication overhead increases with the security level, yet remains practical for the targeted parameters.

In terms of computational overhead, the cycle counts of the C reference implementation grow substantially from NTRE-2593-648 to NTRE-3457-2304, roughly scaling with the increase in n . For example, key generation in C requires **90,555** cycles for NTRE-2593-648, **154,923** cycles for NTRE-2917-1296, and **376,672** cycles for NTRE-3457-2304. The AVX-optimized implementation significantly reduces these costs, bringing key generation down to **56,609**, **81,890**, and **137,912** cycles, respectively. Similarly, encapsulation benefits from optimization, dropping from **72,201** to **58,438** cycles for NTRE-2593-648, from **129,226** to **80,914** cycles for NTRE-2917-1296, and from **351,723** to **171,718** cycles for NTRE-3457-2304.

Overall, NTRE achieves scalable computational performance with moderate communication costs and benefits considerably from low-level AVX2 optimizations, which reduce the cycle counts by a factor of approximately $1.3\times$ to $2.7\times$ across different operations and parameter sets.

5 Statement of Key Features

As a key encapsulation mechanism based on the $\mathcal{R}$ -NTRU $_{\eta}$ and $\mathcal{R}$ -LWE $_{2\eta}$ hard problems, NTRE has the following features:

- **Algorithmic Innovation:** Both NTRE.CPAPKE and NTRE.CCAKEM are newly proposed schemes in this document. We first design the OW-CPA-secure PKE scheme NTRE.CPAPKE, together with a dedicated set of encoding and decoding algorithms (e.g., CBD'_1 for message encoding), as the underlying PKE primitive. We then apply the new PKE-to-KEM transform FOAC' (a slight variant of FOAC) to lift NTRE.CPAPKE to an IND-CCA2-secure KEM, obtaining NTRE.CCAKEM.
- **Algorithmic Simplicity:** Compared with KEMs built from the traditional FO transform, the FOAC' transform used in NTRE.CCAKEM removes the need for the γ -spread condition. This significantly simplifies the design and security analysis of the underlying PKE, without sacrificing CCA security. The scheme still retains a lightweight re-encryption check that adds only minimal overhead but explicitly detects invalid ciphertexts.
- **Algorithmic Flexibility:** Unlike conventional FO-based KEMs that require the underlying PKE to have negligible worst-case decryption failure, NTRE.CCAKEM only requires negligible average-case failure. This relaxed condition allows smaller and more flexible choices of parameters (e.g., $p = 2$ and q as small as 12 bits) while keeping the overall failure probability negligible and the scheme highly efficient.
- **Algorithmic Compatibility:** NTRE is built entirely from standard lattice problems ($\mathcal{R}$ -NTRU $_{\eta}$ and $\mathcal{R}$ -LWE $_{2\eta}$) over the NTT-friendly ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n - X^{n/2} + 1)$. The scheme uses the same type of structured ring as many other post-quantum constructions and can be naturally integrated into existing cryptographic protocols and hybrid solutions.
- **Algorithmic Scalability:** The parameter design of NTRE is highly flexible. By adjusting the polynomial degree n and the modulus q (both constrained by the NTT-friendly condition and security requirements), the scheme can be tuned to different security levels while maintaining an optimal trade-off between communication size and computational cost. The use of $p = 2$ and average-case error further extends the range of practical parameter choices.
- **Implementation Performance Advantages Across Platforms:** The lattice-based structure of NTRE exhibits significant intrinsic parallelism. All core polynomial operations are performed in the NTT domain, and the implementation fully exploits vector instruction sets (AVX2, FMA3, AVX-VNNI) and SHA acceleration. These optimizations provide substantial speed-ups on modern CPUs and can be easily ported to other parallel platforms.

References

- [1] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. Cryptology ePrint Archive, Report 2015/046, 2015.
- [2] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange - A new hope. In Thorsten Holz and Stefan Savage, editors, *USENIX Security 2016*, pages 327–343. USENIX Association, August 2016.
- [3] Andris Ambainis, Mike Hamburg, and Dominique Unruh. Quantum security proofs using semi-classical oracles. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 269–295. Springer, Cham, August 2019.
- [4] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93*, pages 62–73. ACM Press, November 1993.
- [5] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. 2023. available at <http://toc.cryptobook.us/>.
- [6] Cong Chen, Oussama Danba, Jeffrey Hoffstein, Andreas Hulsing, Joost Rijneveld, John M. Schanck, Peter Schwabe, William Whyte, Zhenfei Zhang, Tsunekazu Saito, Takashi Yamakawa, and Keita Xagawa. NTRU. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [7] Jinrong Chen, Rongmao Chen, Yi Wang, Haodong Jiang, Cong Peng, Xinyi Huang, Debiao He, and Xiaofeng Chen. Tighter proofs for PKE-to-KEM transformations under average-case decryption error and without γ -spread. Cryptology ePrint Archive, Paper 2026/468, 2026.
- [8] Jinrong Chen, Yi Wang, Rongmao Chen, Xinyi Huang, and Wei Peng. Tighter proofs for PKE-to-KEM transformation in the quantum random oracle model. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 101–133. Springer, Singapore, December 2024.
- [9] Yuanmi Chen and Phong Q. Nguyen. BKZ 2.0: Better lattice security estimates. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 1–20. Springer, Berlin, Heidelberg, December 2011.
- [10] Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Online-extractability in the quantum random-oracle model. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 677–706. Springer, Cham, May / June 2022.
- [11] Julien Duman, Kathrin Hövelmanns, Eike Kiltz, Vadim Lyubashevsky, Gregor Seiler, and Dominique Unruh. A thorough treatment of highly-efficient NTRU instantiations. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 65–94. Springer, Cham, May 2023.

- [12] Jiangxia Ge, Tianshu Shan, and Rui Xue. Tighter QCCA-secure key encapsulation mechanism with explicit rejection in the quantum random oracle model. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 292–324. Springer, Cham, August 2023.
- [13] Nick Howgrave-Graham. A hybrid lattice-reduction and meet-in-the-middle attack against NTRU. In Alfred Menezes, editor, *CRYPTO 2007*, volume 4622 of *LNCS*, pages 150–169. Springer, Berlin, Heidelberg, August 2007.
- [14] Richard Lindner and Chris Peikert. Better key sizes (and attacks) for LWE-based encryption. In Aggelos Kiayias, editor, *CT-RSA 2011*, volume 6558 of *LNCS*, pages 319–339. Springer, Berlin, Heidelberg, February 2011.
- [15] Vadim Lyubashevsky and Gregor Seiler. NTTRU: Truly fast NTRU using NTT. Cryptology ePrint Archive, Paper 2019/040, 2019.
- [16] National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Technical Report Federal Information Processing Standards Publications (FIPS) 203, U.S. Department of Commerce, Washington, D.C., 2024.
- [17] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information (10th Anniversary edition)*. Cambridge University Press, 2016.
- [18] Jong Hwan Park and Jonghyun Kim. NTRU+: Compact construction of NTRU using simple encoding method, 2026. available at <https://www.ntruplus.org>.
- [19] Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 239–268. Springer, Cham, August 2019.